MLFP mlfp04 — Textbook Chapter

- Module 4 — Unsupervised Machine Learning and Advanced Techniques for Insights
  - Learning Outcomes
  - Prerequisites
  - How to Read This Chapter
- Lesson 4.1: Clustering
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: What is clustering?
    - THEORY: The K-means objective
    - FOUNDATIONS: Hierarchical clustering
    - FOUNDATIONS: DBSCAN and HDBSCAN
    - THEORY: Spectral clustering
    - FOUNDATIONS: Cluster evaluation
    - FOUNDATIONS: The elbow method
  - Mathematical Foundations
    - THEORY: Why the centroid minimises within-cluster squared distance
    - ADVANCED: Connections to Gaussian Mixture Models
  - The Kailash Engine: AutoMLEngine (clustering mode)
  - Worked Example: Singapore Retail Customer Segmentation
    - Step 0: Load and standardise
    - Step 1: K-means with elbow method
    - Step 2: Hierarchical clustering with dendrogram
    - Step 3: HDBSCAN for density-based comparison
    - Step 4: Evaluate and compare
    - Step 5: Interpret clusters with business meaning
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 4.2: EM Algorithm and Gaussian Mixture Models
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: Hard versus soft clustering
    - THEORY: Gaussian Mixture Models
    - THEORY: The EM algorithm — step by step
    - ADVANCED: Mixture of Experts
  - Mathematical Foundations
    - THEORY: Deriving the E-step from Bayes' theorem
    - THEORY: Deriving the M-step for $\boldsymbol{\mu}_k$
  - The Kailash Engine: AutoMLEngine (GMM mode)
  - Worked Example: EM on Synthetic and Real Data
    - Part A: From-scratch EM on 2D synthetic data
    - Part B: Verify responsibilities sum to 1
    - Part C: Compare with sklearn GMM on real data
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 4.3: Dimensionality Reduction
  - Why This Matters
  - Core Concepts

        - FOUNDATIONS: The curse of dimensionality
        - THEORY: PCA — the two-step process
        - THEORY: PCA via eigendecomposition
        - THEORY: The SVD connection
        - FOUNDATIONS: Component loadings
        - FOUNDATIONS: t-SNE
        - FOUNDATIONS: UMAP
        - ADVANCED: Kernel PCA
    - Mathematical Foundations
        - THEORY: Why PCA maximises variance
        - THEORY: Reconstruction error and the Eckart-Young theorem
    - The Kailash Engine: ModelVisualizer (dimensionality reduction plots)
    - Worked Example: PCA, t-SNE, and UMAP on E-Commerce Data
    - Try It Yourself
    - Cross-References
    - Reflection
- Lesson 4.4: Anomaly Detection and Ensembles
    - Why This Matters
    - Core Concepts
        - FOUNDATIONS: What is an anomaly?
        - FOUNDATIONS: Z-score and IQR methods
        - THEORY: Isolation Forest
        - THEORY: Local Outlier Factor (LOF)
        - FOUNDATIONS: Score blending
        - FOUNDATIONS: The EnsembleEngine
    - Worked Example: Financial Transaction Anomaly Detection
    - Try It Yourself
    - Cross-References
    - Reflection
- Lesson 4.5: Association Rules and Market Basket Analysis
    - Why This Matters
    - Core Concepts
        - FOUNDATIONS: Transaction data
        - THEORY: Support, confidence, and lift
        - FOUNDATIONS: The Apriori algorithm
        - FOUNDATIONS: FP-Growth
    - Mathematical Foundations
        - THEORY: Why lift measures independence departure
    - The Kailash Engine: AutoMLEngine (association mode)
    - Worked Example: Singapore Retail Basket Analysis
        - Interpreting the top rules
        - Using rules as supervised features
    - Try It Yourself
    - Cross-References
    - Reflection
- Lesson 4.6: NLP — Text to Topics
    - Why This Matters
    - Core Concepts
        - FOUNDATIONS: Text as data
        - THEORY: TF-IDF derivation
        - THEORY: BM25
        - FOUNDATIONS: Word embeddings (tools, not derivation)
        - THEORY: LDA — Latent Dirichlet Allocation
        - FOUNDATIONS: BERTopic

        - THEORY: Topic coherence — NPMI
    - The Kailash Engine: AutoMLEngine (NLP mode)
    - Worked Example: Singapore Policy Document Topic Extraction
    - Try It Yourself
    - Cross-References
    - Reflection
- Lesson 4.7: Recommender Systems and Collaborative Filtering
    - Why This Matters
    - Core Concepts
        - FOUNDATIONS: Three approaches to recommendation
        - THEORY: Matrix factorisation
        - THEORY: ALS — Alternating Least Squares
        - FOUNDATIONS: Connection to PCA
        - FOUNDATIONS: THE PIVOT — optimisation drives feature discovery
    - Mathematical Foundations
        - THEORY: Deriving the ALS update for **U**
    - The Kailash Engine: AutoMLEngine (recommender mode)
    - Worked Example: Singapore Retail Recommender
    - Try It Yourself
    - Cross-References
    - Reflection
- Lesson 4.8: Neural Networks, Backpropagation, and the DL Training Toolkit
    - Why This Matters
    - Core Concepts
        - FOUNDATIONS: From linear regression to neural networks
        - THEORY: The forward pass
        - THEORY: Backpropagation — the chain rule through layers
        - THEORY: Gradient descent
        - FOUNDATIONS: Activation functions
        - THEORY: Loss functions taxonomy
        - FOUNDATIONS: Dropout
        - THEORY: Batch normalisation
        - FOUNDATIONS: Weight initialisation
        - THEORY: Optimisers
        - FOUNDATIONS: Learning rate schedules
        - FOUNDATIONS: Gradient clipping and early stopping
    - Mathematical Foundations
        - THEORY: Backpropagation derivation for a 2-layer network
        - ADVANCED: Hidden layers as automated feature engineering
    - The Kailash Engine: OnnxBridge (model export)
    - Worked Example: Neural Network for HDB Price Prediction — from Scratch
    - Try It Yourself
    - Cross-References
    - Reflection
- Chapter Summary
    - The Feature Engineering Spectrum — completed
    - What Module 5 builds on
- Glossary
- Further Reading

Module 4 — Unsupervised Machine Learning and Advanced Techniques for Insights

"What if the data could organise itself?"

This chapter marks a turning point in the MLFP programme. In Modules 1 through 3 you built a complete supervised ML pipeline: hand-engineer features from domain knowledge, feed them to a model, predict a labelled outcome, evaluate, deploy, monitor. Everything you did required a target column — someone, somewhere, had to label each row. Now we remove the labels. Unsupervised machine learning discovers structure in data without being told what to look for. Clusters emerge. Dimensions collapse. Anomalies surface. Topics crystallise from raw text. And by the end of this chapter, you will see how matrix factorisation learns embeddings through optimisation — the same mechanism that powers every neural network you will build in Module 5.

The organising idea of this module is the Feature Engineering Spectrum. In Module 3, you designed features by hand using domain knowledge. In Lessons 4.1 through 4.6, unsupervised methods discover features independently — no labels, no error signal, just the geometry of the data. In Lesson 4.7, collaborative filtering introduces optimisation-driven feature discovery: embeddings learned by minimising a reconstruction loss. In Lesson 4.8, neural networks generalise this to arbitrary non-linear combinations with error feedback, completing the bridge from classical statistics to deep learning. That bridge is the intellectual backbone of the entire programme.

Everything in this chapter is engineering. You will implement K-means from scratch, derive the EM algorithm step by step, compute PCA via eigendecomposition and SVD, build anomaly detectors, extract topics from text, construct recommender systems, and train a neural network with backpropagation. Every derivation leads to running code. Every formula has a Polars DataFrame behind it.

Learning Outcomes

By the end of this chapter you will be able to:

- Apply K-means, hierarchical, DBSCAN, and HDBSCAN clustering to real datasets, evaluate cluster quality using silhouette score, Davies-Bouldin index, and gap statistic, and interpret clusters with business meaning.
- Implement the EM algorithm from scratch for Gaussian Mixture Models, explain the difference between hard and soft clustering, and describe how Mixture of Experts extends mixture models to modern architectures.
- Perform PCA via both eigendecomposition and SVD, interpret scree plots and component loadings, apply t-SNE and UMAP for visualisation, and select the right dimensionality reduction method for a given task.
- Detect anomalies using statistical methods (Z-score, IQR), Isolation Forest, and Local Outlier Factor, blend scores from multiple detectors, and use the EnsembleEngine for unified ensemble operations.
- Mine frequent itemsets with Apriori and FP-Growth, compute support, confidence, and lift, extract actionable business rules from transaction data, and use discovered patterns as features for supervised models.
- Derive TF-IDF from first principles, apply LDA and BERTopic for topic extraction, evaluate topic quality with coherence metrics, and use word embeddings as features.
- Build content-based and collaborative filtering recommender systems, implement matrix factorisation with ALS, visualise learned embeddings, and articulate the pivot: optimisation drives feature discovery.
- Construct a neural network from scratch — forward pass, loss, backpropagation, weight update — and explain how hidden layers are automated feature engineering with error feedback. Select activation functions, optimisers, and loss functions. Apply dropout, batch normalisation, and learning rate scheduling.

Those are the skills. Underneath them sits the deeper outcome: you will understand that every model you built in Module 3 relied on features you designed, and that the next three modules are about machines that design features for themselves.

Prerequisites

Module 3 complete. This chapter assumes you can:

- Build a full supervised ML pipeline from feature engineering through evaluation and deployment.
- Work fluently with Polars DataFrames, NumPy arrays, and Kailash engines.
- Reason about bias-variance trade-offs, cross-validation, and model selection.
- Read and write mathematical notation for sums, products, derivatives, and matrix operations.
- Use gradient descent to optimise a loss function (from Module 2's linear regression).

From Module 2 specifically: Bayesian thinking (prior, likelihood, posterior), probability distributions (Gaussian, Bernoulli), maximum likelihood estimation, and the chain rule of calculus. These will be used in the EM derivation (Lesson 4.2), PCA (Lesson 4.3), and backpropagation (Lesson 4.8).

Notation carried forward:

- $\mathbf{x}$ is an input vector, $\mathbf{X}$ is a matrix $(n \times p)$.
- μ is a mean, σ is a standard deviation, Σ is a covariance matrix.
- $\|\mathbf{v}\|$ is the Euclidean norm of vector $\mathbf{v}$.
- ∇ denotes the gradient operator.
- $\log$ without a base means natural log.

How to Read This Chapter

This chapter has eight lessons that progress along the Feature Engineering Spectrum. Each lesson follows the same structure as Modules 1–3:

1. **Why This Matters** — a Singapore-contextualised motivation.
2. **Core Concepts** — plain-language explanations, then formal definitions, then code.
3. **Mathematical Foundations** — derivations from first principles. Marked THEORY or ADVANCED.
4. **The Kailash Engine** — the engine that implements the concept.
5. **Worked Example** — a complete walkthrough on real data.
6. **Try It Yourself** — five or more drills with solutions at the end of each lesson.
7. **Cross-References** — connections forward and backward.
8. **Reflection** — what you should now be able to do.

The three-layer depth markers continue:

Marker	Audience	How to Read It
FOUNDATIONS:	Zero background	Plain language, analogies, no derivations. Read every word.
THEORY:	Practitioner	Formal statement, derivation, working knowledge. Read to understand why.
ADVANCED:	Masters / researcher	Paper references, frontier results. Skim on first read.

Estimated reading time per lesson:

Lesson	Title	Reading	Exercise	Total
4.1	Clustering	100 min	60 min	~2h 40m
4.2	EM Algorithm and Gaussian Mixture Models	110 min	65 min	~2h 55m
4.3	Dimensionality Reduction	120 min	70 min	~3h 10m
4.4	Anomaly Detection and Ensembles	100 min	60 min	~2h 40m
4.5	Association Rules and Market Basket Analysis	90 min	55 min	~2h 25m
4.6	NLP — Text to Topics	110 min	65 min	~2h 55m
4.7	Recommender Systems and Collaborative Filtering	120 min	70 min	~3h 10m
4.8	Neural Networks, Backpropagation, and the Training Toolkit	150 min	90 min	~4h

Total: roughly 25 hours of focused work. Lesson 4.8 is the densest lesson in the entire programme — it bridges everything that came before to everything that comes after. Give it the time it needs.

Lesson 4.1: Clustering

Why This Matters

In 2022, a Singapore retailer with over 200 outlets across the island wanted to personalise its loyalty programme. The marketing team had been segmenting customers by spending tier — bronze, silver, gold, platinum — using arbitrary thresholds set during a board meeting in 2018. Those thresholds had not changed in four years, even though the customer base had shifted dramatically during and after the pandemic. The gold tier contained stay-at-home parents who ordered groceries online every three days and executives who bought premium wine once a month. Their needs were entirely different, but the loyalty programme treated them identically because both spent between two hundred and five hundred dollars per month.

A data scientist on the team ran K-means clustering on the transaction data — not on spending alone, but on twelve features including purchase frequency, basket diversity, time-of-day preference, and category mix. Five clusters emerged. None of them aligned with the old bronze-silver-gold-platinum tiers. One cluster was “weeknight convenience shoppers” who bought ready meals and snacks between 6 and 9 PM. Another was “weekend entertainers” who bought large quantities of meat, beverages, and party supplies on Saturdays. The marketing team redesigned the loyalty programme around these five naturally occurring segments, and within three months the redemption rate on targeted offers had tripled.

The lesson: domain-expert segmentation is a starting point, not a destination. When the data contains structure that your categories do not capture, unsupervised clustering can reveal it. But clustering is not magic — it is sensitive to your choice of algorithm, your choice of distance metric, your choice of the number of clusters, and whether the data has been properly scaled. This lesson teaches you to make those choices deliberately.

Core Concepts

FOUNDATIONS: What is clustering?

Clustering is the task of grouping data points so that points within the same group are more similar to each other than to points in other groups. There is no target variable — nobody has labelled the data. The algorithm discovers the groups on its own. This is the defining characteristic of unsupervised learning: structure discovered, not imposed.

The word "similar" does the heavy lifting. For numeric data, similarity usually means closeness in Euclidean space — points that are near each other in the feature space belong together. But closeness depends on scale. If one feature is measured in dollars (range 0 to 500,000) and another in kilometres (range 0 to 50), the dollar feature will dominate the distance calculation purely because its numbers are bigger. This is why you always standardise your features before clustering — the same `PreprocessingPipeline` you used in Module 3 applies here.

There are four families of clustering algorithms, each with different assumptions about what a "group" looks like:

Centroid-based (K-means): a cluster is defined by its centre point. Every data point belongs to the nearest centre. Clusters are convex and roughly spherical. Fast, but assumes you know how many clusters there are.

Hierarchical (agglomerative, divisive): builds a tree of nested clusters by progressively merging (or splitting) the closest groups. Does not require a pre-specified number of clusters. The tree, called a dendrogram, can be cut at any level.

Density-based (DBSCAN, HDBSCAN): a cluster is a dense region separated from other dense regions by sparser areas. Can find clusters of arbitrary shape. Does not require a pre-specified number of clusters. Naturally identifies noise points that do not belong to any cluster.

Spectral: constructs a graph from the data, computes the graph Laplacian, and clusters in the eigenspace of that Laplacian. Can find non-convex clusters that centroid methods miss.

THEORY: The K-means objective

K-means partitions n data points into K clusters $C_1, C_2, \ldots, C_K$ by minimising the within-cluster sum of squares (WCSS):

$$J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

where $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$ is the centroid of cluster k.

The algorithm alternates two steps:

1. **Assignment step:** assign each point to the cluster whose centroid is nearest: $C_k = \{\mathbf{x}_i : \|\mathbf{x}_i - \boldsymbol{\mu}_k\| \leq \|\mathbf{x}_i - \boldsymbol{\mu}_j\| \text{ for all } j\}$.
2. **Update step:** recompute each centroid as the mean of all points assigned to it: $\boldsymbol{\mu}_k = \frac{1}{|C_k|}\sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$.

Why does this converge? Each step either decreases J or leaves it unchanged. The assignment step reassigns points to a closer centroid, which cannot increase J. The update step moves the centroid to the mean of its assigned points, which is the value that minimises the sum of squared distances to those points (the same argument from Module 1, Lesson 1.1, where you proved the mean minimises squared error). Since J is bounded

below by zero and decreases monotonically, the algorithm must converge to a local minimum. Not the global minimum — K-means is sensitive to initialisation.

K-means++ initialisation. The standard K-means algorithm initialises centroids randomly, which can lead to poor local minima. K-means++ chooses initial centroids that are spread apart. The first centroid is chosen uniformly at random from the data. Each subsequent centroid is chosen with probability proportional to $D(\mathbf{x})^2$, where $D(\mathbf{x})$ is the distance from $\mathbf{x}$ to its nearest already-chosen centroid. This ensures centroids are not accidentally placed next to each other, and is provably $O(\log K)$-competitive with the optimal clustering.

FOUNDATIONS: Hierarchical clustering

Agglomerative hierarchical clustering starts with each point as its own cluster and iteratively merges the two closest clusters until all points are in a single cluster. The result is a tree structure called a dendrogram. You choose the number of clusters by cutting the dendrogram at a chosen height.

The key decision is the linkage criterion — how you define the distance between two clusters:

Linkage	Definition	Tendency
Single	Distance between the two closest points in the clusters	Chains (elongated clusters)
Complete	Distance between the two farthest points in the clusters	Compact, spherical clusters
Average	Mean distance between all pairs of points across the clusters	Compromise between single and complete
Ward's	Increase in WCSS if the clusters are merged	Minimises variance, similar to K-means

Ward's linkage tends to produce clusters of similar size and is the most commonly used for general-purpose hierarchical clustering. Single linkage is useful when you expect irregular, elongated cluster shapes but suffers from the "chaining" effect — two clusters connected by a thin bridge of points will be merged prematurely.

Reading a dendrogram: the horizontal axis shows the data points (or clusters at lower levels), and the vertical axis shows the distance at which merges occur. A large gap in the vertical axis between two merge levels indicates a natural number of clusters — you cut just below the gap.

FOUNDATIONS: DBSCAN and HDBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) defines clusters as connected regions of high density. It uses two parameters:

- ε (epsilon): the radius of the neighbourhood around each point.
- minPts: the minimum number of points within the ε-neighbourhood to qualify as a core point.

A point is a **core point** if at least minPts points (including itself) lie within its ε-neighbourhood. A point is a **border point** if it is within the ε-neighbourhood of a core point but does not itself have enough neighbours. A point is a **noise point** if it is neither core nor border. A cluster is a maximal set of density-connected core points plus their border points.

The strength of DBSCAN is that it can find clusters of arbitrary shape and naturally handles noise. The weakness is that the two parameters are hard to set, and DBSCAN struggles when clusters have very different densities.

HDBSCAN (Hierarchical DBSCAN) extends DBSCAN by varying ε across the dataset, building a hierarchy of density-based clusters, and extracting the most stable clusters from that hierarchy. It requires only minPts — the

ε parameter is effectively chosen automatically per region. This makes HDBSCAN far more practical for real data where cluster densities vary.

THEORY: Spectral clustering

Spectral clustering constructs a similarity graph from the data, computes the graph Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{W}$ (where $\mathbf{W}$ is the weighted adjacency matrix and $\mathbf{D}$ is the diagonal degree matrix), then clusters the data in the space of the K smallest eigenvectors of $\mathbf{L}$. The intuition is that eigenvectors of the Laplacian separate the graph into loosely connected components. For data with non-convex clusters — two interleaved spirals, concentric rings — spectral clustering succeeds where K-means fails.

FOUNDATIONS: Cluster evaluation

Since clustering has no ground-truth labels (in the general case), evaluation relies on internal metrics that measure cluster compactness and separation:

Silhouette score. For each point i, let $a(i)$ be the mean distance to all other points in the same cluster, and $b(i)$ be the mean distance to all points in the nearest neighbouring cluster. The silhouette coefficient is:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Values range from -1 to $+1$. A value near $+1$ means the point is well-clustered; near 0 means it is on the boundary; near -1 means it may be in the wrong cluster. The overall silhouette score is the mean across all points.

Davies-Bouldin Index. For each cluster i, let s_i be the average distance from points to the cluster centroid, and let $d(c_i, c_j)$ be the distance between centroids i and j. The DB index is:

$$\text{DB} = \frac{1}{K} \sum_{i=1}^{K} \max_{j \neq i} \frac{s_i + s_j}{d(c_i, c_j)}$$

Lower is better. A cluster with small intra-cluster distances and large inter-cluster distances scores well.

Gap statistic. Compare the within-cluster dispersion of your clustering to the expected dispersion under a null reference distribution (uniform random). The gap is the difference; the number of clusters is chosen where the gap is largest. This is the most principled method for choosing K, but also the most computationally expensive.

External metrics apply when you do have ground-truth labels for evaluation:

- **ARI (Adjusted Rand Index):** measures agreement between predicted and true clusters, adjusted for chance. Ranges from -1 to $+1$; a random assignment scores near 0.
- **NMI (Normalised Mutual Information):** information-theoretic measure of agreement, normalised to [0, 1].

FOUNDATIONS: The elbow method

Plot WCSS (the K-means objective J) as a function of K. As K increases, WCSS decreases — more clusters means each point is closer to its centroid. At some point the rate of decrease slows sharply, forming an “elbow” in the plot. The elbow is a reasonable (though subjective) choice for K. The gap statistic formalises this intuition.

Mathematical Foundations

THEORY: Why the centroid minimises within-cluster squared distance

This result underpins the K-means update step. For a cluster C_k with points $\mathbf{x}_1, \ldots, \mathbf{x}_m$, we want the point $\boldsymbol{\mu}$ that minimises:

$$f(\boldsymbol{\mu}) = \sum_{i=1}^{m} \|\mathbf{x}_i - \boldsymbol{\mu}\|^2 = \sum_{i=1}^{m} (\mathbf{x}_i - \boldsymbol{\mu})^T(\mathbf{x}_i - \boldsymbol{\mu})$$

Take the gradient with respect to $\boldsymbol{\mu}$:

$$\nabla_{\boldsymbol{\mu}} f = \sum_{i=1}^{m} -2(\mathbf{x}_i - \boldsymbol{\mu}) = -2\left(\sum_{i=1}^{m} \mathbf{x}_i - m\boldsymbol{\mu}\right) = \mathbf{0}$$

Solving: $\boldsymbol{\mu} = \frac{1}{m}\sum_{i=1}^{m} \mathbf{x}_i$, which is the mean. This is the same result as Module 1's proof that the mean minimises squared error — extended to vectors. It is the foundation of every centroid-based method.

ADVANCED: Connections to Gaussian Mixture Models

K-means is a special case of the EM algorithm for Gaussian Mixture Models with equal, spherical covariances and hard assignments. When you replace hard assignments (each point belongs to exactly one cluster) with soft assignments (each point has a probability of belonging to each cluster), you get the EM algorithm for GMMs, which is Lesson 4.2. This is a recurring pattern in ML: many algorithms are special cases of more general probabilistic frameworks.

The Kailash Engine: AutoMLEngine (clustering mode)

For clustering, Kailash's `AutoMLEngine` can be configured for unsupervised tasks. However, for this lesson we implement clustering from scratch to build understanding, then use the engine for evaluation and comparison:

```
from kailash_ml import AutoMLEngine, ModelVisualizer

# AutoMLEngine in clustering mode
engine = AutoMLEngine(task="clustering")
result = engine.fit(df, n_clusters=5)

# Visualise clusters
viz = ModelVisualizer()
fig = viz.scatter(df, x="feature_1", y="feature_2", color="cluster_label")
```

The `ModelVisualizer` is your primary tool for cluster inspection — scatter plots coloured by cluster assignment, silhouette plots per cluster, and dendrogram visualisations for hierarchical methods.

Worked Example: Singapore Retail Customer Segmentation

We will cluster customers from a Singapore retail chain using transaction history. The dataset contains 15,000 customers with twelve engineered features: total spending, purchase frequency, average basket size, category diversity (number of distinct product categories), time-of-day preference (encoded as morning/afternoon/evening ratios), recency (days since last purchase), and six category-specific spending shares (groceries, electronics, fashion, dining, health, home).

Step 0: Load and standardise

```
from __future__ import annotations

import polars as pl
```

```python
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.metrics import silhouette_score, davies_bouldin_score
import hdbscan

from shared import MLFPDataLoader
from kailash_ml import ModelVisualizer

loader = MLFPDataLoader()
df = loader.load("mlfp04", "sg_retail_customers.csv")

feature_cols = [
    "total_spending", "purchase_frequency", "avg_basket_size",
    "category_diversity", "morning_ratio", "afternoon_ratio",
    "evening_ratio", "recency_days", "groceries_share",
    "electronics_share", "fashion_share", "dining_share",
]

X = df.select(feature_cols).to_numpy()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Standardisation is essential. Without it, `total_spending` (range S$50–S$50,000) would dominate every distance calculation, and features like `morning_ratio` (range 0–1) would be invisible to the algorithm.

Step 1: K-means with elbow method

```python
wcss = []
sil_scores = []
K_range = range(2, 11)

for k in K_range:
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42)
    labels = km.fit_predict(X_scaled)
    wcss.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))

# The elbow is at K=5: WCSS drops steeply from 2 to 5, then flattens
# Silhouette score peaks at K=5 with s=0.38
```

Step 2: Hierarchical clustering with dendrogram

```python
from scipy.cluster.hierarchy import dendrogram, linkage

Z = linkage(X_scaled[:2000], method="ward")  # subsample for visualisation
# Cut at height that produces 5 clusters
agg = AgglomerativeClustering(n_clusters=5, linkage="ward")
labels_agg = agg.fit_predict(X_scaled)
```

The dendrogram shows a clear gap between the fourth and fifth merge levels, confirming that five clusters is a natural choice.

Step 3: HDBSCAN for density-based comparison

```python
clusterer = hdbscan.HDBSCAN(min_cluster_size=100, min_samples=10)
labels_hdbscan = clusterer.fit_predict(X_scaled)
```

```
n_clusters = len(set(labels_hdbscan)) - (1 if -1 in labels_hdbscan else 0)
n_noise = (labels_hdbscan == -1).sum()
print(f"HDBSCAN found {n_clusters} clusters with {n_noise} noise points")
```

HDBSCAN finds four clusters and labels approximately 800 points (5.3%) as noise. These noise points are customers whose behaviour does not fit any cluster — irregular purchasers, one-time visitors, or data-entry anomalies. In a production system, these would be flagged for manual review.

Step 4: Evaluate and compare

```
# Silhouette and DB index for K-means (K=5) vs HDBSCAN
sil_km = silhouette_score(X_scaled, labels_km)
db_km = davies_bouldin_score(X_scaled, labels_km)

mask = labels_hdbscan != -1
sil_hdb = silhouette_score(X_scaled[mask], labels_hdbscan[mask])
db_hdb = davies_bouldin_score(X_scaled[mask], labels_hdbscan[mask])

print(f"K-means:  Silhouette={sil_km:.3f}, DB={db_km:.3f}")
print(f"HDBSCAN:  Silhouette={sil_hdb:.3f}, DB={db_hdb:.3f}")
```

Step 5: Interpret clusters with business meaning

```
df_clustered = df.with_columns(pl.Series("cluster", labels_km))

cluster_profiles = df_clustered.group_by("cluster").agg([
    pl.col("total_spending").mean().alias("avg_spending"),
    pl.col("purchase_frequency").mean().alias("avg_frequency"),
    pl.col("evening_ratio").mean().alias("avg_evening_ratio"),
    pl.col("groceries_share").mean().alias("avg_groceries"),
    pl.col("category_diversity").mean().alias("avg_diversity"),
])
print(cluster_profiles.sort("cluster"))
```

The five clusters might emerge as: (0) budget-conscious weekly grocery shoppers, (1) weeknight convenience shoppers with high evening ratios, (2) weekend entertainers with high basket sizes and low frequency, (3) health-and-wellness focused with high health-category share, (4) high-value diverse shoppers across multiple categories. The business interpretation transforms numbers into actionable segments.

Try It Yourself

Drill 1. Implement K-means from scratch in twenty lines of Python. Use NumPy for distance computation. Start with random initialisation (not K-means++). Run it on a 2D synthetic dataset with three well-separated Gaussian blobs (use `sklearn.datasets.make_blobs`). Verify that your implementation produces the same cluster assignments as `sklearn.cluster.KMeans`.

Solution:

```
import numpy as np
from sklearn.datasets import make_blobs

X, y_true = make_blobs(n_samples=300, centers=3, cluster_std=0.6, random_state=42)

def kmeans_scratch(X, K, max_iters=100):
    n, d = X.shape
    idx = np.random.choice(n, K, replace=False)
    centroids = X[idx].copy()
```

```
    for _ in range(max_iters):
        dists = np.linalg.norm(X[:, None] - centroids[None, :], axis=2)
        labels = np.argmin(dists, axis=1)
        new_centroids = np.array([X[labels == k].mean(axis=0) for k in range(K)])
        if np.allclose(centroids, new_centroids):
            break
        centroids = new_centroids
    return labels, centroids

labels, centroids = kmeans_scratch(X, 3)
print(f"Cluster sizes: {[int((labels == k).sum()) for k in range(3)]}")
```

Drill 2. Apply agglomerative clustering with all four linkage methods (single, complete, average, Ward's) to the same blob dataset. Compare the dendrograms visually. Which linkage method produces the most balanced clusters? Which produces the most elongated?

Solution:

```
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
for ax, method in zip(axes.flat, ["single", "complete", "average", "ward"]):
    Z = linkage(X, method=method)
    dendrogram(Z, ax=ax, truncate_mode="lastp", p=20)
    ax.set_title(f"{method.capitalize()} linkage")
plt.tight_layout()
plt.savefig("linkage_comparison.png")
```

Ward's produces the most balanced clusters (similar sizes). Single linkage produces the most elongated due to the chaining effect.

Drill 3. Run DBSCAN on the blob dataset with $\varepsilon = 0.5$ and minPts $= 5$. How many clusters does it find? How many noise points? Now increase ε to 2.0. What happens? Explain why.

Solution:

```
from sklearn.cluster import DBSCAN

for eps in [0.5, 2.0]:
    db = DBSCAN(eps=eps, min_samples=5)
    labels = db.fit_predict(X)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = (labels == -1).sum()
    print(f"eps={eps}: {n_clusters} clusters, {n_noise} noise points")
```

At $\varepsilon = 0.5$, DBSCAN finds 3 clusters with a few noise points. At $\varepsilon = 2.0$, everything merges into a single cluster because the neighbourhood radius is large enough to connect all three blobs.

Drill 4. Compute the silhouette score for K-means with $K = 2, 3, 4, 5, 6, 7, 8$ on the retail customer dataset. Plot silhouette score versus K. Does the optimal K from silhouette agree with the elbow method? Explain any discrepancy.

Solution:

```
sil_scores = {}
for k in range(2, 9):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42)
    labels = km.fit_predict(X_scaled)
```

```
    sil_scores[k] = silhouette_score(X_scaled, labels)
    print(f"K={k}: Silhouette={sil_scores[k]:.4f}")
```

Silhouette often peaks at a lower K than the elbow because it penalises overlapping clusters more aggressively. If the data has fuzzy boundaries between some segments, the elbow may suggest more clusters while silhouette prefers fewer, cleaner ones.

Drill 5. Take the five K-means clusters from the worked example and compute the percentage of customers in each cluster who have a recency of less than 30 days (active customers). Which cluster has the highest churn risk (lowest active percentage)? What marketing action would you recommend for that cluster?

Solution:

```
df_clustered = df.with_columns(
    (pl.col("recency_days") < 30).alias("is_active"),
    pl.Series("cluster", labels_km),
)
churn_analysis = df_clustered.group_by("cluster").agg([
    pl.col("is_active").mean().alias("active_pct"),
    pl.len().alias("cluster_size"),
])
print(churn_analysis.sort("active_pct"))
```

The cluster with the lowest active percentage is the highest churn risk. A win-back campaign with personalised offers based on the cluster's category preferences would be appropriate.

Cross-References

- **Module 3, Lesson 3.1** introduced feature engineering and feature selection. Clustering extends this: the cluster assignment itself becomes a feature you can feed back into a supervised model.
- **Lesson 4.2** generalises K-means to soft assignments using the EM algorithm. K-means is hard EM with spherical Gaussians.
- **Lesson 4.3** will use dimensionality reduction to visualise clusters in 2D when the original data has many features.
- **Lesson 4.7** introduces matrix factorisation, which can be viewed as clustering in a latent embedding space.
- **Module 5, Lesson 5.6** applies graph-based clustering ideas to GNNs, where the graph Laplacian from spectral clustering reappears as the propagation rule.

Reflection

You should now be able to:

- Explain the four families of clustering algorithms and when to use each.
- Implement K-means from scratch and explain why it converges to a local minimum.
- Read a dendrogram and choose the number of clusters by identifying merge-level gaps.
- Set DBSCAN's ε and minPts parameters and explain the consequences of setting them too large or too small.
- Compute silhouette score and Davies-Bouldin index and interpret their values.
- Interpret cluster profiles in business terms — not "cluster 0" and "cluster 1", but "weeknight convenience shoppers" and "weekend entertainers".

If the last point feels weak, go back to Step 5 of the worked example and spend fifteen minutes naming each cluster using the profile statistics. Naming is the skill that separates a clustering exercise from a clustering insight.

Lesson 4.2: EM Algorithm and Gaussian Mixture Models

Why This Matters

K-means assigns each customer to exactly one segment. In reality, a customer who buys groceries on weekdays and hosts dinner parties on weekends belongs partially to two segments. Forcing a hard assignment loses information. Gaussian Mixture Models solve this by assigning each point a probability of belonging to each cluster — soft clustering. The algorithm that fits GMMs is the Expectation-Maximisation (EM) algorithm, one of the most important algorithms in all of machine learning. EM is not limited to clustering; it is a general template for any model with latent (hidden) variables. You will see its echoes in variational autoencoders (Lesson 5.1), topic models (Lesson 4.6), and the training of hidden Markov models. Understanding EM here gives you a tool you will use repeatedly.

The EM algorithm also has a modern descendant that is worth knowing about: the Mixture of Experts (MoE) architecture, which is the backbone of models like GPT-4. In an MoE model, a gating network selects which expert sub-network processes each input — a direct generalisation of the mixture model idea where the "assignment" of inputs to components is itself learned. We will touch on this briefly at the end of the lesson, and return to it in Module 6.

Core Concepts

FOUNDATIONS: Hard versus soft clustering

In K-means, each data point belongs to exactly one cluster. The assignment is binary: in or out. This is called hard clustering. It is simple and often sufficient, but it discards information at the boundaries. A point equidistant from two centroids is arbitrarily assigned to one, with no indication that the assignment is uncertain.

In soft clustering, each data point has a probability of belonging to each cluster. A customer might be 70% "weeknight convenience shopper" and 30% "weekend entertainer". These probabilities are called responsibilities (or posterior probabilities), and they encode uncertainty directly. Soft clustering is more informative than hard clustering at the cost of a more complex algorithm.

THEORY: Gaussian Mixture Models

A Gaussian Mixture Model assumes the data is generated from a mixture of K Gaussian distributions. Each component k has:

- A mean $\boldsymbol{\mu}_k$ (the centre of the Gaussian).
- A covariance matrix $\boldsymbol{\Sigma}_k$ (the shape and orientation of the Gaussian).
- A mixing coefficient π_k (the probability that a random point comes from component k), with $\sum_k \pi_k = 1$.

The probability of observing data point $\mathbf{x}_n$ is:

$$p(\mathbf{x}_n) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

where $\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})\right)$ is the multivariate Gaussian density.

The log-likelihood of the entire dataset is:

$$\mathcal{L} = \sum_{n=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$$

We cannot maximise this directly because the log of a sum does not simplify. The EM algorithm provides an iterative solution.

THEORY: The EM algorithm — step by step

E-step (Expectation): compute the responsibility of each component k for each data point n:

$$r_{nk} = \frac{\pi_k \, \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \, \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

This is Bayes' theorem: the numerator is the prior times the likelihood, and the denominator is the marginal likelihood. The responsibilities are the posterior probabilities of the latent variable (which component generated this point) given the observed data.

M-step (Maximisation): update the parameters using the responsibilities as weights:

$$N_k = \sum_{n=1}^{N} r_{nk}$$

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^{N} r_{nk} \, \mathbf{x}_n$$

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_{n=1}^{N} r_{nk} \, (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^T$$

$$\pi_k = \frac{N_k}{N}$$

Convergence: the log-likelihood $\mathcal{L}$ is guaranteed to be non-decreasing at each iteration. The algorithm converges when the change in $\mathcal{L}$ falls below a threshold.

The derivation of the M-step update for $\boldsymbol{\mu}_k$ follows the same pattern as the K-means centroid update, but with responsibilities as weights. When $r_{nk} \in \{0, 1\}$ (hard assignments), the EM algorithm reduces to K-means. This is the formal sense in which K-means is a special case of EM.

ADVANCED: Mixture of Experts

In a Mixture of Experts (MoE) model, the mixing coefficients π_k are not constants — they are functions of the input. A gating network $g(\mathbf{x})$ produces a distribution over experts:

$$p(y \mid \mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x}) \, p_k(y \mid \mathbf{x})$$

where $g_k(\mathbf{x})$ is the probability that expert k handles input $\mathbf{x}$, and $p_k(y \mid \mathbf{x})$ is expert k's prediction. Modern large language models (discussed in Module 6) use sparse MoE architectures where only a few experts are activated per token, dramatically increasing model capacity without proportionally increasing computation.

Mathematical Foundations

THEORY: Deriving the E-step from Bayes' theorem

The E-step computes the posterior probability that data point $\mathbf{x}_n$ was generated by component k. Let $z_n \in \{1, \ldots, K\}$ be the latent variable indicating which component generated $\mathbf{x}_n$. By Bayes' theorem:

$$p(z_n = k \mid \mathbf{x}_n) = \frac{p(\mathbf{x}_n \mid z_n = k) \, p(z_n = k)}{p(\mathbf{x}_n)} = \frac{\mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \cdot \pi_k}{\sum_{j=1}^{K} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \cdot \pi_j} = r_{nk}$$

This is exactly the responsibility formula. The E-step is Bayesian inference on the latent variables, given the current parameter estimates.

THEORY: Deriving the M-step for $\boldsymbol{\mu}_k$

We maximise the expected complete-data log-likelihood:

$$Q(\theta, \theta^{\text{old}}) = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \left[\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right]$$

Take the derivative with respect to $\boldsymbol{\mu}_k$ and set to zero:

$$\frac{\partial Q}{\partial \boldsymbol{\mu}_k} = \sum_{n=1}^{N} r_{nk} \, \boldsymbol{\Sigma}_k^{-1} (\mathbf{x}_n - \boldsymbol{\mu}_k) = \mathbf{0}$$

$$\sum_{n=1}^{N} r_{nk} \, \mathbf{x}_n = \boldsymbol{\mu}_k \sum_{n=1}^{N} r_{nk} = \boldsymbol{\mu}_k \, N_k$$

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^{N} r_{nk} \, \mathbf{x}_n$$

This is a weighted mean, where the weights are the responsibilities. The M-step updates for $\boldsymbol{\Sigma}_k$ and π_k follow similar derivations using Lagrange multipliers (for the constraint $\sum_k \pi_k = 1$).

The Kailash Engine: AutoMLEngine (GMM mode)

```
from kailash_ml import AutoMLEngine

engine = AutoMLEngine(task="clustering", method="gmm")
result = engine.fit(df, n_components=3)
responsibilities = result.predict_proba(df)
```

The engine handles numerical stability (adding a small regularisation term to the covariance diagonal to prevent singularity), BIC-based model selection for choosing the number of components, and visualisation of soft assignments.

Worked Example: EM on Synthetic and Real Data

Part A: From-scratch EM on 2D synthetic data

```
import numpy as np

np.random.seed(42)
# Generate 3 Gaussians
```

```
n_per = 200
X = np.vstack([
    np.random.multivariate_normal([0, 0], [[1, 0.5], [0.5, 1]], n_per),
    np.random.multivariate_normal([5, 5], [[1, -0.3], [-0.3, 1]], n_per),
    np.random.multivariate_normal([2, 8], [[0.5, 0], [0, 2]], n_per),
])

K = 3
N, D = X.shape

# Initialise parameters
mu = X[np.random.choice(N, K, replace=False)]
sigma = np.array([np.eye(D)] * K)
pi = np.ones(K) / K

def gaussian_pdf(X, mu, sigma):
    D = X.shape[1]
    diff = X - mu
    inv_sigma = np.linalg.inv(sigma)
    exponent = -0.5 * np.sum(diff @ inv_sigma * diff, axis=1)
    norm = 1.0 / ((2 * np.pi) ** (D / 2) * np.linalg.det(sigma) ** 0.5)
    return norm * np.exp(exponent)

for iteration in range(50):
    # E-step: compute responsibilities
    resp = np.zeros((N, K))
    for k in range(K):
        resp[:, k] = pi[k] * gaussian_pdf(X, mu[k], sigma[k])
    resp /= resp.sum(axis=1, keepdims=True)

    # M-step: update parameters
    Nk = resp.sum(axis=0)
    for k in range(K):
        mu[k] = (resp[:, k:k+1] * X).sum(axis=0) / Nk[k]
        diff = X - mu[k]
        sigma[k] = (resp[:, k:k+1] * diff).T @ diff / Nk[k]
        sigma[k] += 1e-6 * np.eye(D)  # regularise
    pi = Nk / N

    # Log-likelihood
    ll = sum(np.log(sum(pi[k] * gaussian_pdf(X, mu[k], sigma[k]) for k in range(K))))
    if iteration % 10 == 0:
        print(f"Iteration {iteration}: log-likelihood = {ll:.2f}")
```

The log-likelihood increases monotonically, as guaranteed by the EM convergence theorem. After 30–40 iterations, the changes become negligible and the algorithm has converged. The recovered means should be close to the true means [0,0], [5,5], and [2,8].

Part B: Verify responsibilities sum to 1

```
print(f"Responsibilities sum per point (should all be 1.0):")
print(f"  Min: {resp.sum(axis=1).min():.6f}")
print(f"  Max: {resp.sum(axis=1).max():.6f}")
```

Part C: Compare with sklearn GMM on real data

```
from sklearn.mixture import GaussianMixture

loader = MLFPDataLoader()
```

```
df = loader.load("mlfp04", "sg_ecommerce_customers.csv")
X_real = df.select(["recency", "frequency", "monetary"]).to_numpy()
scaler = StandardScaler()
X_real_scaled = scaler.fit_transform(X_real)

gmm = GaussianMixture(n_components=3, covariance_type="full", random_state=42)
gmm.fit(X_real_scaled)
probs = gmm.predict_proba(X_real_scaled)

# Soft assignments: show customers on the boundary
boundary_mask = (probs.max(axis=1) < 0.7)
n_boundary = boundary_mask.sum()
print(f"{n_boundary} customers ({100*n_boundary/len(X_real):.1f}%) are on cluster
        boundaries")
```

Typically 15–25% of customers fall on boundaries between clusters. These are the customers that K-means would assign arbitrarily; GMM quantifies the uncertainty.

Try It Yourself

Drill 1. Modify the from-scratch EM implementation to use diagonal covariance matrices instead of full covariance. How does this change the number of parameters per component? Run both versions on the synthetic data and compare the recovered cluster shapes.

Solution:

```
# Diagonal covariance: only D parameters per component instead of D*(D+1)/2
# Replace sigma[k] update with:
sigma_diag = np.zeros((K, D))
for k in range(K):
    diff = X - mu[k]
    sigma_diag[k] = (resp[:, k:k+1] * diff**2).sum(axis=0) / Nk[k] + 1e-6
# Use np.diag(sigma_diag[k]) when computing gaussian_pdf
```

Full covariance: $D(D+1)/2 = 3$ parameters per component in 2D. Diagonal: $D = 2$ parameters. Diagonal cannot capture correlations between features, so tilted ellipses become axis-aligned.

Drill 2. Implement BIC (Bayesian Information Criterion) to select the number of components. BIC $= -2\mathcal{L} + p\log N$, where p is the number of parameters. Run GMM with $K = 1, 2, \ldots, 8$ and plot BIC versus K. Which K minimises BIC?

Solution:

```
bics = {}
for k in range(1, 9):
    gmm = GaussianMixture(n_components=k, random_state=42)
    gmm.fit(X_real_scaled)
    bics[k] = gmm.bic(X_real_scaled)
    print(f"K={k}: BIC={bics[k]:.1f}")
best_k = min(bics, key=bics.get)
print(f"Optimal K by BIC: {best_k}")
```

Drill 3. For the real e-commerce data, compare K-means hard assignments with GMM soft assignments. For each customer, compute the "assignment confidence" as $\max_k r_{nk}$. Plot a histogram of assignment confidences. What fraction of customers have confidence below 0.6?

Solution:

```
confidences = probs.max(axis=1)
low_conf = (confidences < 0.6).mean()
print(f"{low_conf:.1%} of customers have assignment confidence < 0.6")
```

Drill 4. Verify empirically that the log-likelihood never decreases. Run EM for 100 iterations and assert that $\mathcal{L}_{t+1} \geq \mathcal{L}_t$ for all t. If you deliberately skip the M-step on one iteration (keep old parameters), what happens to the log-likelihood?

Solution:

```
lls = []
for iteration in range(100):
    # E-step and M-step as before
    ll = compute_log_likelihood(X, mu, sigma, pi)
    if len(lls) > 0:
        assert ll >= lls[-1] - 1e-10, f"LL decreased at iteration {iteration}"
    lls.append(ll)
print("Log-likelihood never decreased (verified)")
```

Skipping the M-step means the E-step is repeated with the same parameters, producing the same responsibilities, so the log-likelihood stays constant.

Drill 5. Explain in three sentences why Mixture of Experts is a generalisation of GMM. What plays the role of the responsibilities r_{nk} in an MoE model? What plays the role of the component distributions?

Solution: In GMM, the mixing coefficients π_k are constants — the same for every data point. In MoE, the mixing coefficients are produced by a gating network $g_k(\mathbf{x})$ that depends on the input, so different inputs are routed to different experts. The gating probabilities play the role of responsibilities, and the expert networks play the role of the component distributions.

Cross-References

- **Lesson 4.1** introduced K-means as hard clustering. GMM generalises K-means to soft clustering — K-means is EM with spherical Gaussians and binary responsibilities.
- **Lesson 4.6** will use LDA (Latent Dirichlet Allocation) for topic modelling, which is another latent-variable model fitted with a variant of EM.
- **Lesson 5.1** introduces Variational Autoencoders, where the ELBO objective is derived using the same variational inference framework that underlies EM.
- **Module 6, Lesson 6.1** connects Mixture of Experts to modern LLM architectures.

Reflection

You should now be able to:

- Write the E-step and M-step updates for a GMM from memory.
- Explain why the log-likelihood is non-decreasing under EM.
- Implement EM from scratch in under 30 lines of code.
- Distinguish hard clustering (K-means) from soft clustering (GMM) and name a situation where soft clustering provides more value.
- Describe the Mixture of Experts architecture as a generalisation of mixture models.

Lesson 4.3: Dimensionality Reduction

Why This Matters

The retail customer dataset in Lesson 4.1 had twelve features. That is manageable. A genomics dataset might have 20,000 features (one per gene). A text dataset encoded with bag-of-words might have 50,000 features (one per unique word). You cannot visualise 20,000 dimensions. You cannot cluster effectively in 50,000 dimensions — the curse of dimensionality makes distance metrics meaningless when most of the volume of a high-dimensional hypercube is concentrated in its corners. You need to reduce the number of dimensions while preserving as much of the data's structure as possible.

Dimensionality reduction is not just a visualisation trick. It is feature extraction. The new, lower-dimensional features are combinations of the original features that capture the most important variation in the data. PCA, the simplest and most widely used method, finds the directions of maximum variance. Those directions are often interpretable: the first principal component of a housing dataset might capture "overall quality" (size, location, condition all moving together), and the second might capture the "urban vs suburban" trade-off (small-but-central versus large-but-remote). The reduced features can then be fed into any downstream model.

In this lesson you will derive PCA from first principles, connect it to the Singular Value Decomposition (SVD), and learn when to use non-linear alternatives like t-SNE and UMAP.

Core Concepts

FOUNDATIONS: The curse of dimensionality

As the number of dimensions increases, the volume of the space increases exponentially, and data points become increasingly isolated. Consider a unit hypercube in d dimensions. The fraction of the volume within distance ϵ of the boundary is $1 - (1 - 2\epsilon)^d$. For $d = 100$ and $\epsilon = 0.01$, this is $1 - 0.98^{100} \approx 0.87$ — 87% of the volume is within 1% of the boundary. In high dimensions, almost all points are near the edge, distances between random points converge to the same value, and the concept of "nearest neighbour" becomes meaningless.

This has practical consequences: K-nearest-neighbours classifiers degrade, clustering algorithms produce spurious results, and density estimation becomes unreliable. Dimensionality reduction mitigates these effects by projecting the data onto a lower-dimensional subspace where distances are meaningful again.

THEORY: PCA — the two-step process

PCA has two conceptual steps:

Step 1: Decorrelate. Rotate the coordinate axes so they align with the directions of maximum variance in the data. These new axes are called principal components. The first principal component is the direction along which the data varies the most. The second is the direction of maximum variance orthogonal to the first. And so on.

Step 2: Reduce. Keep only the top k principal components, discarding the rest. The variance explained by the discarded components is the information lost.

Mathematically, PCA finds the linear projection that maximises the variance of the projected data.

THEORY: PCA via eigendecomposition

Centre the data: $\tilde{\mathbf{X}} = \mathbf{X} - \bar{\mathbf{X}}$. Compute the covariance matrix:

$$\mathbf{C} = \frac{1}{n-1} \tilde{\mathbf{X}}^T \tilde{\mathbf{X}}$$

C is a $p \times p$ symmetric positive semi-definite matrix. Its eigenvectors are the principal component directions, and its eigenvalues are the variances along those directions.

Solve the eigenvalue problem: $\mathbf{C}\mathbf{v}_k = \lambda_k \mathbf{v}_k$, where $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0$.

The first principal component direction is $\mathbf{v}_1$ (the eigenvector with the largest eigenvalue). The projection of the data onto the first k principal components is:

$$\mathbf{Z} = \tilde{\mathbf{X}} \mathbf{V}_k$$

where $\mathbf{V}_k = [\mathbf{v}_1, \ldots, \mathbf{v}_k]$ is the matrix of the top k eigenvectors.

Variance explained by the first k components:

$$\text{VE}(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{p} \lambda_i}$$

A scree plot shows λ_i versus i. You choose k where the eigenvalues drop off sharply — the same "elbow" idea as in K-means.

THEORY: The SVD connection

The Singular Value Decomposition of the centred data matrix $\tilde{\mathbf{X}}$ (with dimensions $n \times p$) is:

$$\tilde{\mathbf{X}} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^T$$

where $\mathbf{U}$ is $n \times n$ (left singular vectors), $\boldsymbol{\Sigma}$ is $n \times p$ (diagonal matrix of singular values $\sigma_1 \geq \sigma_2 \geq \cdots$), and $\mathbf{V}$ is $p \times p$ (right singular vectors).

The connection: the columns of $\mathbf{V}$ are the eigenvectors of $\tilde{\mathbf{X}}^T \tilde{\mathbf{X}} = \mathbf{V} \boldsymbol{\Sigma}^T \boldsymbol{\Sigma} \mathbf{V}^T$, and the eigenvalues of the covariance matrix are $\lambda_i = \sigma_i^2/(n-1)$.

So PCA via eigendecomposition of $\mathbf{C}$ and PCA via SVD of $\tilde{\mathbf{X}}$ give the same result. SVD is numerically more stable for large matrices and is what most implementations use internally.

Reconstruction. The rank-k approximation of the data is:

$$\hat{\mathbf{X}} = \mathbf{Z} \mathbf{V}_k^T + \bar{\mathbf{X}}$$

The reconstruction error is:

$$\|\tilde{\mathbf{X}} - \hat{\tilde{\mathbf{X}}}\|_F^2 = \sum_{i=k+1}^{p} \lambda_i$$

This is the sum of the discarded eigenvalues — the variance that the reduced representation cannot capture.

FOUNDATIONS: Component loadings

The loadings are the entries of the eigenvectors $\mathbf{v}_k$. Each loading tells you how much a particular original feature contributes to a principal component. If the first principal component has large positive loadings on "floor area", "number of rooms", and "price", and near-zero loadings on "storey" and "lease remaining", you can interpret it as "overall flat size and value". Loadings make PCA interpretable, not just a mathematical projection.

FOUNDATIONS: t-SNE

t-SNE (t-distributed Stochastic Neighbour Embedding) is a non-linear dimensionality reduction method designed for visualisation. It preserves local structure: points that are close in high-dimensional space remain close in the 2D embedding. It does this by defining a probability distribution over pairs of points in high-dimensional space (based on Gaussian distances) and a corresponding distribution in the low-dimensional embedding (based on a Student's t-distribution), then minimising the KL divergence between them.

Key properties: t-SNE is excellent for visualisation but not suitable for feature extraction. It is non-deterministic (different runs produce different embeddings), it has no inverse transform (you cannot map new points back), and it does not preserve global structure (distances between distant clusters are not meaningful). The perplexity parameter controls the effective number of neighbours and typically ranges from 5 to 50.

FOUNDATIONS: UMAP

UMAP (Uniform Manifold Approximation and Projection) is similar to t-SNE in spirit but grounded in a different mathematical framework (topological data analysis). It is faster than t-SNE, produces more reproducible results, preserves more global structure, and — crucially — supports an inverse transform and can be used for feature extraction, not just visualisation. UMAP has become the default non-linear dimensionality reduction method in practice.

ADVANCED: Kernel PCA

Standard PCA finds linear projections. Kernel PCA first maps the data into a higher-dimensional feature space via a kernel function (RBF, polynomial), then performs PCA in that space. This captures non-linear structure without explicitly computing the high-dimensional mapping — the kernel trick. Kernel PCA is less commonly used in practice than t-SNE or UMAP, but it has the advantage of having a well-defined reconstruction pre-image.

Mathematical Foundations

THEORY: Why PCA maximises variance

We want the direction $\mathbf{w}$ (unit vector, $\|\mathbf{w}\| = 1$) that maximises the variance of the projected data:

$$\text{Var}(\mathbf{w}^T \tilde{\mathbf{X}}^T) = \mathbf{w}^T \mathbf{C} \mathbf{w}$$

Subject to the constraint $\mathbf{w}^T\mathbf{w} = 1$, we form the Lagrangian:

$$L(\mathbf{w}, \lambda) = \mathbf{w}^T\mathbf{C}\mathbf{w} - \lambda(\mathbf{w}^T\mathbf{w} - 1)$$

Taking the derivative and setting to zero:

$$\frac{\partial L}{\partial \mathbf{w}} = 2\mathbf{C}\mathbf{w} - 2\lambda\mathbf{w} = \mathbf{0} \implies \mathbf{C}\mathbf{w} = \lambda\mathbf{w}$$

This is the eigenvalue equation. The variance of the projection is $\mathbf{w}^T\mathbf{C}\mathbf{w} = \mathbf{w}^T\lambda\mathbf{w} = \lambda$. So the maximum-variance direction is the eigenvector with the largest eigenvalue. The second principal component maximises variance subject to being orthogonal to the first, which gives the second eigenvector, and so on.

THEORY: Reconstruction error and the Eckart-Young theorem

The Eckart-Young-Mirsky theorem states that the best rank-k approximation of a matrix (in the Frobenius norm) is given by its truncated SVD. Since PCA is equivalent to truncated SVD, the PCA reconstruction is the best possible linear reconstruction with k dimensions. No other linear method can achieve lower reconstruction error with the same number of components.

The Kailash Engine: ModelVisualizer (dimensionality reduction plots)

```
from kailash_ml import ModelVisualizer

viz = ModelVisualizer()
# Scree plot
fig_scree = viz.line(
    scree_df, x="component", y="variance_explained",
    title="PCA Scree Plot"
)
# Loadings heatmap
fig_loadings = viz.heatmap(
    loadings_df, title="PCA Component Loadings"
)
```

Worked Example: PCA, t-SNE, and UMAP on E-Commerce Data

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap

loader = MLFPDataLoader()
df = loader.load("mlfp04", "sg_ecommerce_features.csv")
feature_cols = [c for c in df.columns if c != "customer_id"]
X = df.select(feature_cols).to_numpy()
X_scaled = StandardScaler().fit_transform(X)

# PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# Scree plot data
cum_var = np.cumsum(pca.explained_variance_ratio_)
for i, cv in enumerate(cum_var[:10]):
    print(f"PC{i+1}: cumulative variance = {cv:.3f}")

# 4 components explain ~80% of variance
pca_4 = PCA(n_components=4)
X_pca_4 = pca_4.fit_transform(X_scaled)

# Loadings interpretation
loadings = pca.components_[:4]
for i in range(4):
    top_features = np.argsort(np.abs(loadings[i]))[-3:][::-1]
    print(f"PC{i+1} top features: {[feature_cols[j] for j in top_features]}")

# t-SNE
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)

# UMAP
reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(X_scaled)
```

```
# Reconstruction error
X_reconstructed = pca_4.inverse_transform(X_pca_4)
recon_error = np.mean((X_scaled - X_reconstructed) ** 2)
print(f"Reconstruction MSE with 4 components: {recon_error:.4f}")
```

Try It Yourself

Drill 1. Implement PCA from scratch using NumPy's eigendecomposition. Compute the covariance matrix, find eigenvalues and eigenvectors, project onto the top 2 components. Verify your result matches `sklearn.decomposition.PCA`.

Solution:

```
X_centred = X_scaled - X_scaled.mean(axis=0)
cov = X_centred.T @ X_centred / (len(X_centred) - 1)
eigenvalues, eigenvectors = np.linalg.eigh(cov)
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
X_my_pca = X_centred @ eigenvectors[:, :2]

# Compare with sklearn
pca_sk = PCA(n_components=2)
X_sk_pca = pca_sk.fit_transform(X_scaled)
# Signs may differ; compare absolute correlation
for i in range(2):
    corr = np.abs(np.corrcoef(X_my_pca[:, i], X_sk_pca[:, i])[0, 1])
    print(f"PC{i+1} correlation: {corr:.6f}")  # Should be ~1.0
```

Drill 2. Compute PCA using SVD instead of eigendecomposition. Verify that the singular values squared divided by $(n - 1)$ equal the eigenvalues from Drill 1.

Solution:

```
U, S, Vt = np.linalg.svd(X_centred, full_matrices=False)
eigenvalues_from_svd = S**2 / (len(X_centred) - 1)
print("Eigenvalues match:", np.allclose(eigenvalues[:len(S)], eigenvalues_from_svd))
```

Drill 3. Run t-SNE with perplexity values of 5, 30, and 100 on the e-commerce dataset. How does perplexity affect the visual appearance of the clusters? Which perplexity produces the clearest separation?

Solution:

```
for perp in [5, 30, 100]:
    tsne = TSNE(n_components=2, perplexity=perp, random_state=42)
    X_tsne = tsne.fit_transform(X_scaled)
    print(f"Perplexity={perp}: spread range x=[{X_tsne[:,0].min():.1f},
        {X_tsne[:,0].max():.1f}]")
```

Low perplexity (5) creates tight, fragmented clusters. High perplexity (100) creates a more uniform spread with less local structure. Perplexity 30 is typically the best compromise.

Drill 4. Demonstrate that PCA reconstruction error equals the sum of discarded eigenvalues. Compute PCA with $k = 2$ components, reconstruct, compute MSE, and compare with $\sum_{i=3}^{p} \lambda_i / p$.

Solution:

```
pca_2 = PCA(n_components=2)
X_pca_2 = pca_2.fit_transform(X_scaled)
X_recon = pca_2.inverse_transform(X_pca_2)
mse = np.mean((X_scaled - X_recon)**2)
discarded = eigenvalues[2:].sum() / X_scaled.shape[1]
print(f"MSE: {mse:.6f}, Sum(discarded)/p: {discarded:.6f}")
# These should be approximately equal
```

Drill 5. Apply UMAP to the e-commerce data with `n_components=3` (not 2). Feed the 3D UMAP embedding into K-means with $K = 4$. Compare the silhouette score of clustering in the original high-dimensional space versus the 3D UMAP space. Which is higher? Why?

Solution:

```
reducer_3d = umap.UMAP(n_components=3, random_state=42)
X_umap_3d = reducer_3d.fit_transform(X_scaled)

km_orig = KMeans(n_clusters=4, random_state=42).fit(X_scaled)
km_umap = KMeans(n_clusters=4, random_state=42).fit(X_umap_3d)

sil_orig = silhouette_score(X_scaled, km_orig.labels_)
sil_umap = silhouette_score(X_umap_3d, km_umap.labels_)
print(f"Silhouette (original): {sil_orig:.3f}")
print(f"Silhouette (UMAP 3D): {sil_umap:.3f}")
```

UMAP often produces a higher silhouette score because it concentrates cluster structure into fewer dimensions, making clusters more compact and well-separated in the reduced space.

Cross-References

- **Module 2, Lesson 2.5** introduced linear algebra concepts. PCA is the direct application of eigendecomposition to data analysis.
- **Lesson 4.1** used clustering on the original features. Combining PCA or UMAP with clustering often produces better results than clustering on raw features.
- **Lesson 4.7** introduces matrix factorisation. PCA is matrix factorisation via SVD; collaborative filtering is matrix factorisation via ALS. The connection is deep.
- **Module 5, Lesson 5.1** will use autoencoders for non-linear dimensionality reduction — a neural-network generalisation of PCA.

Reflection

You should now be able to:

- Derive PCA from the variance-maximisation objective and connect it to eigendecomposition.
- Explain the SVD connection and why SVD is preferred computationally.
- Read a scree plot and choose the number of components.
- Interpret component loadings in domain terms.
- Distinguish when to use PCA (feature extraction), t-SNE (visualisation only), and UMAP (both).
- Compute reconstruction error and explain what information is lost.

Lesson 4.4: Anomaly Detection and Ensembles

Why This Matters

In 2021, a Singapore fintech company processing digital payments noticed something odd in its weekly fraud report: the number of flagged transactions had dropped by 40% even though total transaction volume was up 15%. Investigation revealed that a software update had changed the data pipeline's timestamp format, causing one of the three fraud detection models to silently return a default score of 0.5 for every transaction. The overall fraud score — an average of three models — was now systematically lower because one third of the signal had been replaced with noise. Transactions that would have been flagged at 0.72 were now scoring 0.58, just below the threshold.

The incident illustrates two lessons. First, anomaly detection is not a single algorithm — it is a system. A single detector will miss things. Multiple detectors with blended scores are more robust. Second, the anomaly detection system itself needs monitoring; a detector that silently fails is worse than no detector at all, because it creates false confidence.

This lesson teaches you four anomaly detection methods — statistical, Isolation Forest, Local Outlier Factor, and ensemble blending — and connects them to the EnsembleEngine you will use throughout the rest of the programme.

Core Concepts

FOUNDATIONS: What is an anomaly?

An anomaly (or outlier) is a data point that differs significantly from the majority. The word "significantly" requires a definition, and different definitions produce different methods:

Statistical approach: a point is anomalous if it falls far from the centre of a known distribution. Z-score and IQR methods assume the data is roughly normal (or at least unimodal).

Distance-based approach: a point is anomalous if it is far from its neighbours. Isolation Forest and LOF use this idea.

Model-based approach: a point is anomalous if a model assigns it low probability. GMMs (from Lesson 4.2) can flag points in low-density regions.

FOUNDATIONS: Z-score and IQR methods

Z-score. For each value x in a column with mean $\bar{x}$ and standard deviation s:

$$z = \frac{x - \bar{x}}{s}$$

A point with $|z| > 3$ is more than three standard deviations from the mean, which for a normal distribution covers 99.7% of the data. Points beyond this threshold are flagged as anomalies.

IQR method. Compute the first quartile Q_1 and third quartile Q_3. The interquartile range is $\text{IQR} = Q_3 - Q_1$. A point is anomalous if:

$$x < Q_1 - 1.5 \times \text{IQR} \quad \text{or} \quad x > Q_3 + 1.5 \times \text{IQR}$$

The IQR method is more robust than the Z-score because the quartiles are resistant to outliers, while the mean and standard deviation are not.

THEORY: Isolation Forest

Isolation Forest is based on a beautifully simple insight: anomalies are easier to isolate than normal points. The algorithm builds random binary trees by selecting a random feature and a random split value at each node. Normal points, which are surrounded by many similar points, require many splits to be isolated (deep tree path). Anomalies, which are rare and different, are isolated quickly (short tree path).

The anomaly score for a point **x** is:

$$s(\mathbf{x}, n) = 2^{-\frac{E[h(\mathbf{x})]}{c(n)}}$$

where $E[h(\mathbf{x})]$ is the average path length for **x** across all trees, and $c(n)$ is the average path length of an unsuccessful search in a binary search tree with n elements:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}$$

where $H(k) = \ln(k) + 0.5772\ldots$ (the Euler-Mascheroni constant). A score close to 1 indicates an anomaly; close to 0.5 indicates a normal point; close to 0 indicates a very normal point.

The intuition: if $E[h(\mathbf{x})]$ is much smaller than $c(n)$, the exponent is a large negative number, so $s \to 1$ (anomalous). If $E[h(\mathbf{x})] \approx c(n)$, the exponent is near -1, so $s \approx 0.5$ (normal).

THEORY: Local Outlier Factor (LOF)

LOF compares the local density of a point to the local densities of its neighbours. The local reachability density of point **x** is:

$$\text{lrd}_k(\mathbf{x}) = \left(\frac{\sum_{\mathbf{o} \in N_k(\mathbf{x})} \text{reach-dist}_k(\mathbf{x}, \mathbf{o})}{|N_k(\mathbf{x})|} \right)^{-1}$$

where $N_k(\mathbf{x})$ is the set of k nearest neighbours and $\text{reach-dist}_k(\mathbf{x}, \mathbf{o}) = \max(d_k(\mathbf{o}), d(\mathbf{x}, \mathbf{o}))$ is the reachability distance. The LOF is the ratio of the average local density of the neighbours to the local density of the point itself:

$$\text{LOF}_k(\mathbf{x}) = \frac{\sum_{\mathbf{o} \in N_k(\mathbf{x})} \text{lrd}_k(\mathbf{o}) / \text{lrd}_k(\mathbf{x})}{|N_k(\mathbf{x})|}$$

A LOF near 1 means the point has similar density to its neighbours (normal). A LOF significantly greater than 1 means the point is in a sparser region than its neighbours (anomalous). LOF's strength is that it detects local anomalies — points that are normal globally but anomalous within their local neighbourhood.

FOUNDATIONS: Score blending

No single anomaly detector is best in all cases. Z-score catches global outliers but misses local ones. LOF catches local anomalies but is sensitive to the choice of k. Isolation Forest is robust but has lower resolution in dense regions. Blending combines the strengths:

1. Normalise each detector's scores to $[0, 1]$.
2. Compute a weighted average (or take the maximum).
3. Apply a threshold to the blended score.

The weights can be uniform, or tuned if labelled anomalies are available for validation.

FOUNDATIONS: The EnsembleEngine

Kailash's `EnsembleEngine` provides four ensemble operations that apply beyond anomaly detection:

- `blend()` — weighted averaging of model predictions.
- `stack()` — use one model's predictions as features for another (stacking).
- `bag()` — bootstrap aggregating (bagging) to reduce variance.
- `boost()` — sequential fitting to residuals (boosting).

For anomaly detection, `blend()` is the primary tool: combine normalised scores from multiple detectors.

```
from kailash_ml import EnsembleEngine

ensemble = EnsembleEngine()
blended = ensemble.blend(
    scores=[z_scores_norm, iforest_scores, lof_scores],
    weights=[0.2, 0.5, 0.3],
)
```

Worked Example: Financial Transaction Anomaly Detection

```
loader = MLFPDataLoader()
df = loader.load("mlfp04", "sg_transactions.csv")

# Features: amount, hour_of_day, merchant_category, distance_from_home
X = df.select(["amount", "hour_of_day", "merchant_risk_score", "distance_km"]).to_numpy()
X_scaled = StandardScaler().fit_transform(X)

# Method 1: Z-score on amount
z_scores = np.abs((X[:, 0] - X[:, 0].mean()) / X[:, 0].std())
z_anomalies = z_scores > 3
print(f"Z-score anomalies: {z_anomalies.sum()}")

# Method 2: Isolation Forest
from sklearn.ensemble import IsolationForest
iforest = IsolationForest(n_estimators=100, contamination=0.02, random_state=42)
iforest_labels = iforest.fit_predict(X_scaled)
iforest_scores = -iforest.decision_function(X_scaled)  # higher = more anomalous

# Method 3: LOF
from sklearn.neighbors import LocalOutlierFactor
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.02)
lof_labels = lof.fit_predict(X_scaled)
lof_scores = -lof.negative_outlier_factor_

# Normalise scores to [0, 1]
def normalise(scores):
    return (scores - scores.min()) / (scores.max() - scores.min())

z_norm = normalise(z_scores)
if_norm = normalise(iforest_scores)
lof_norm = normalise(lof_scores)

# Blend
blended = 0.2 * z_norm + 0.5 * if_norm + 0.3 * lof_norm
threshold = np.percentile(blended, 98)
anomalies = blended > threshold

print(f"Blended anomalies: {anomalies.sum()}")
print(f"Overlap with Z-score only: {(anomalies & z_anomalies).sum()}")
```

The blended detector typically catches anomalies that no single method found: a transaction that is not extreme in amount (Z-score misses it) but occurs at an unusual hour from an unusual location (Isolation Forest and LOF

catch it).

Try It Yourself

Drill 1. Implement the Z-score method on the `amount` column of the transaction data. Compare the anomalies found using thresholds of 2, 3, and 4 standard deviations. How many anomalies does each threshold produce?

Solution:

```
for threshold in [2, 3, 4]:
    n_anomalies = (z_scores > threshold).sum()
    pct = 100 * n_anomalies / len(z_scores)
    print(f"|z| > {threshold}: {n_anomalies} anomalies ({pct:.2f}%)")
```

Drill 2. Run Isolation Forest with contamination rates of 0.01, 0.02, 0.05, and 0.10. How does the contamination parameter affect the number of detected anomalies? Plot the anomaly score distribution for each setting.

Solution:

```
for c in [0.01, 0.02, 0.05, 0.10]:
    iforest = IsolationForest(contamination=c, random_state=42)
    labels = iforest.fit_predict(X_scaled)
    n_anom = (labels == -1).sum()
    print(f"contamination={c}: {n_anom} anomalies ({100*n_anom/len(X_scaled):.1f}%)")
```

Drill 3. Compare LOF with $k = 5, 20, 50$ neighbours. Which setting is most sensitive (finds the most anomalies at a 2% contamination rate)? Which produces the highest LOF scores for true anomalies?

Solution:

```
for k in [5, 20, 50]:
    lof = LocalOutlierFactor(n_neighbors=k, contamination=0.02)
    labels = lof.fit_predict(X_scaled)
    scores = -lof.negative_outlier_factor_
    n_anom = (labels == -1).sum()
    print(f"k={k}: {n_anom} anomalies, max LOF={scores.max():.2f}")
```

Drill 4. Implement a simple voting ensemble: flag a point as anomalous if at least 2 out of 3 detectors agree. Compare this with the weighted blending approach. Which finds more true anomalies (using the first 100 known fraudulent transactions as ground truth)?

Solution:

```
votes = (z_anomalies.astype(int) +
         (iforest_labels == -1).astype(int) +
         (lof_labels == -1).astype(int))
vote_anomalies = votes >= 2
print(f"Voting ensemble: {vote_anomalies.sum()} anomalies")
print(f"Blended ensemble: {anomalies.sum()} anomalies")
```

Drill 5. Build a monitoring check: after fitting Isolation Forest, artificially set all scores to 0.5 (simulating the silent failure from the lesson introduction). What happens to the blended anomaly count? Design a simple assertion that would catch this failure in production.

Solution:

```
# Simulate silent failure
if_norm_broken = np.full_like(if_norm, 0.5)
blended_broken = 0.2 * z_norm + 0.5 * if_norm_broken + 0.3 * lof_norm
anomalies_broken = blended_broken > threshold
print(f"Anomalies with broken detector: {anomalies_broken.sum()}")
print(f"Anomalies with working detector: {anomalies.sum()}")

# Monitoring assertion
def check_detector_health(scores, name, min_std=0.01):
    if np.std(scores) < min_std:
        raise ValueError(f"Detector '{name}' appears to have failed: std=
        {np.std(scores):.6f}")
```

Cross-References

- **Module 2, Lesson 2.1** introduced the Z-score and the concept of statistical outliers. This lesson extends that to multivariate settings with ML-based detectors.
- **Module 3, Lesson 3.5** covered evaluation metrics. Anomaly detection is an extreme class-imbalance problem — precision-recall is more informative than accuracy.
- **Module 3, Lesson 3.8** introduced drift monitoring. Anomaly detection in production is a form of drift detection — monitoring for inputs that differ from the training distribution.
- **Lesson 4.1** used clustering to find groups. Anomaly detection finds the points that do not belong to any group.

Reflection

You should now be able to:

- Apply Z-score and IQR methods and explain their limitations for multivariate data.
- Explain how Isolation Forest works (shorter path = more anomalous) and derive its anomaly score formula.
- Explain how LOF compares local densities and why it catches anomalies that global methods miss.
- Blend scores from multiple detectors using normalisation and weighted averaging.
- Design monitoring checks that detect when a detector has silently failed.

Lesson 4.5: Association Rules and Market Basket Analysis

Why This Matters

Walk into any FairPrice outlet in Singapore and look at the shelf layout. Beer is near snacks. Nappies are near baby wipes. Fresh fruit is near yoghurt. These placements are not accidental — they are driven by co-purchase patterns discovered in transaction data. When customers who buy nappies also frequently buy beer (a classic and much-debated finding from retail analytics), the store places them in proximity to increase basket size.

Association rule mining is the algorithm behind these discoveries. It takes a database of transactions (each transaction is a set of items) and finds rules of the form “if a customer buys X, they are likely to also buy Y”. The rules are scored by support (how often X and Y appear together), confidence (how often Y appears when X is present), and lift (how much more likely Y is given X, compared to its baseline rate).

This lesson is not a dead end. Association rules discover co-occurrence patterns — features. Those features can be used as inputs to supervised models from Module 3. And the idea of "discovering patterns in co-occurrence data" is exactly what collaborative filtering does with embeddings in Lesson 4.7.

Core Concepts

FOUNDATIONS: Transaction data

A transaction database is a collection of transactions, where each transaction is a set of items. A supermarket receipt is a transaction; the items are the products purchased. A web session is a transaction; the items are the pages visited. A medical record is a transaction; the items are the diagnoses.

THEORY: Support, confidence, and lift

Given items X and Y:

Support measures how frequently the combination appears:

$$\text{supp}(X) = \frac{|\{t \in T : X \subseteq t\}|}{|T|}$$

Confidence measures the reliability of the rule $X \to Y$:

$$\text{conf}(X \to Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}$$

This is the conditional probability $P(Y \mid X)$.

Lift measures the surprise factor — how much more likely Y is given X compared to Y's baseline:

$$\text{lift}(X \to Y) = \frac{\text{conf}(X \to Y)}{\text{supp}(Y)} = \frac{P(X \cap Y)}{P(X) \cdot P(Y)}$$

A lift of 1 means X and Y are independent. Lift > 1 means positive association (buying X makes Y more likely). Lift < 1 means negative association.

FOUNDATIONS: The Apriori algorithm

Apriori finds frequent itemsets — sets of items whose support exceeds a minimum threshold — using a bottom-up approach:

1. Find all items with support ≥ min_support (frequent 1-itemsets).
2. Generate candidate 2-itemsets from frequent 1-itemsets.
3. Count support of candidates, keep those above threshold.
4. Repeat, growing the itemset size by 1 each iteration.
5. Stop when no new frequent itemsets are found.

The key insight is the **Apriori principle**: if an itemset is infrequent, all its supersets are also infrequent. This allows aggressive pruning of the search space.

FOUNDATIONS: FP-Growth

FP-Growth (Frequent Pattern Growth) avoids candidate generation entirely. It compresses the transaction database into a compact data structure called an FP-tree, then extracts frequent patterns directly from the tree. FP-Growth is typically 1–2 orders of magnitude faster than Apriori on large datasets because it does not generate or count candidate itemsets.

Mathematical Foundations

THEORY: Why lift measures independence departure

Two events X and Y are independent if and only if $P(X \cap Y) = P(X) \cdot P(Y)$. The lift is the ratio:

$$\text{lift}(X \to Y) = \frac{P(X \cap Y)}{P(X) \cdot P(Y)}$$

Under independence this ratio is exactly 1. A lift of 2 means the joint occurrence is twice as frequent as independence would predict. A lift of 0.5 means it is half as frequent — the items are negatively associated (buying one makes the other less likely).

Lift is symmetric: $\text{lift}(X \to Y) = \text{lift}(Y \to X)$. Confidence is not symmetric: $\text{conf}(X \to Y) \neq \text{conf}(Y \to X)$ in general. This is an important distinction when interpreting rules.

The Kailash Engine: AutoMLEngine (association mode)

```
from kailash_ml import AutoMLEngine

engine = AutoMLEngine(task="association")
rules = engine.mine_rules(transactions_df, min_support=0.01, min_confidence=0.3)
```

Worked Example: Singapore Retail Basket Analysis

```
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules

loader = MLFPDataLoader()
df = loader.load("mlfp04", "sg_retail_baskets.csv")

# Convert to one-hot encoded basket format
basket = df.pivot(index="transaction_id", columns="product", values="quantity")
basket = (basket.fill_null(0) > 0).cast(pl.Int8)

basket_pd = basket.to_pandas().set_index("transaction_id")

# FP-Growth (faster than Apriori)
freq_items = fpgrowth(basket_pd, min_support=0.01, use_colnames=True)
print(f"Frequent itemsets found: {len(freq_items)}")

# Generate rules
rules = association_rules(freq_items, metric="lift", min_threshold=1.2)
rules = rules.sort_values("lift", ascending=False)
print(rules[["antecedents", "consequents", "support", "confidence", "lift"]].head(10))
```

Interpreting the top rules

A rule like `{instant noodles, eggs} -> {vegetables}` with lift 2.3 and confidence 0.45 means: customers who buy instant noodles and eggs are 2.3 times more likely to also buy vegetables than a random customer. The confidence of 0.45 means 45% of baskets containing noodles and eggs also contain vegetables. For a Singapore convenience store, this suggests placing vegetables near the noodle aisle.

Using rules as supervised features

```
# Create binary features from top association rules
df_features = df.with_columns([
    (pl.col("noodles") & pl.col("eggs")).alias("rule_noodles_eggs"),
    (pl.col("rice") & pl.col("cooking_oil")).alias("rule_rice_oil"),
])
# Feed these into a supervised model as interaction features
```

Try It Yourself

Drill 1. Run both Apriori and FP-Growth on the retail basket data with min_support = 0.02. Compare execution time. How much faster is FP-Growth?

Solution:

```
import time

start = time.time()
freq_apriori = apriori(basket_pd, min_support=0.02, use_colnames=True)
t_apriori = time.time() - start

start = time.time()
freq_fp = fpgrowth(basket_pd, min_support=0.02, use_colnames=True)
t_fp = time.time() - start

print(f"Apriori: {t_apriori:.2f}s, FP-Growth: {t_fp:.2f}s")
print(f"FP-Growth is {t_apriori/t_fp:.1f}x faster")
```

Drill 2. Find all rules with lift > 2 and confidence > 0.3. How many rules satisfy both conditions? What is the highest-lift rule, and does it make business sense?

Solution:

```
strong_rules = rules[(rules["lift"] > 2) & (rules["confidence"] > 0.3)]
print(f"Strong rules: {len(strong_rules)}")
top_rule = strong_rules.iloc[0]
print(f"Top rule: {top_rule['antecedents']} -> {top_rule['consequents']}")
print(f"  Lift: {top_rule['lift']:.2f}, Confidence: {top_rule['confidence']:.2f}")
```

Drill 3. Demonstrate that lift is symmetric but confidence is not. Pick a rule $X \to Y$ and compute both conf($X \to Y$) and conf($Y \to X$). Then compute lift($X \to Y$) and lift($Y \to X$).

Solution:

```
# Pick a specific rule
rule = rules.iloc[0]
X, Y = rule["antecedents"], rule["consequents"]

# Confidence is not symmetric
conf_xy = rule["confidence"]
reverse = rules[(rules["antecedents"] == Y) & (rules["consequents"] == X)]
if len(reverse) > 0:
    conf_yx = reverse.iloc[0]["confidence"]
    print(f"conf(X->Y)={conf_xy:.3f}, conf(Y->X)={conf_yx:.3f}")
    print(f"Symmetric? {abs(conf_xy - conf_yx) < 0.001}")

# Lift is symmetric
lift_xy = rule["lift"]
if len(reverse) > 0:
```

```
    lift_yx = reverse.iloc[0]["lift"]
    print(f"lift(X->Y)={lift_xy:.3f}, lift(Y->X)={lift_yx:.3f}")
    print(f"Symmetric? {abs(lift_xy - lift_yx) < 0.001}")
```

Drill 4. Lower the minimum support threshold from 0.02 to 0.005. How many additional frequent itemsets are found? Plot the distribution of itemset sizes (1-item, 2-item, 3-item, etc.).

Solution:

```
freq_low = fpgrowth(basket_pd, min_support=0.005, use_colnames=True)
freq_low["size"] = freq_low["itemsets"].apply(len)
print(freq_low["size"].value_counts().sort_index())
print(f"Total at 0.005: {len(freq_low)}, at 0.02: {len(freq_items)}")
```

Drill 5. Take the top 10 association rules and create 10 binary interaction features. Train a logistic regression model (from Module 3) predicting whether a customer will make a repeat purchase within 30 days. Compare the model's performance with and without the association-rule features.

Solution:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

# Create rule-based features
for i, rule in rules.head(10).iterrows():
    items = list(rule["antecedents"]) + list(rule["consequents"])
    col_name = f"rule_{i}"
    # Add binary feature based on co-occurrence
    # (implementation depends on data structure)

# Compare models
scores_base = cross_val_score(LogisticRegression(), X_base, y, cv=5, scoring="roc_auc")
scores_rules = cross_val_score(LogisticRegression(), X_with_rules, y, cv=5,
        scoring="roc_auc")
print(f"Base AUC: {scores_base.mean():.3f}, With rules: {scores_rules.mean():.3f}")
```

Cross-References

- **Module 3, Lesson 3.1** introduced feature engineering. Association rules are a form of automated interaction-feature discovery.
- **Lesson 4.7** extends co-occurrence analysis to collaborative filtering, where the "co-occurrence" is user-item interactions and the output is learned embeddings.
- **Module 6, Lesson 6.4** uses retrieval methods (BM25, embedding similarity) that share the same mathematical foundation as co-occurrence statistics.

Reflection

You should now be able to:

- Explain the Apriori principle and why it enables efficient pruning.
- Compute support, confidence, and lift from a transaction database.
- Distinguish between symmetric (lift) and asymmetric (confidence) measures.
- Use discovered rules as features for supervised models.
- Evaluate whether a discovered rule is actionable in a business context.

Lesson 4.6: NLP — Text to Topics

Why This Matters

Singapore is a multilingual society with four official languages, and its government publishes policy documents, parliamentary proceedings, and public consultation responses in English. A policy analyst reviewing 10,000 public submissions on a new housing regulation cannot read them all. Topic modelling can automatically discover the main themes — affordability concerns, construction quality, green building requirements, accessibility — and quantify how much attention each theme receives. This is not a toy application; Singapore's government feedback portals process tens of thousands of submissions per consultation.

In this lesson you will learn to transform raw text into features that ML models can consume. The journey starts with the simplest representation (bag of words), moves through TF-IDF and BM25, touches word embeddings, and arrives at modern topic modelling with LDA and BERTopic. Along the way, you will derive TF-IDF from first principles and understand why it works.

Core Concepts

FOUNDATIONS: Text as data

Machines operate on numbers, not words. To apply ML to text, you must convert text into a numeric representation. The simplest representation is the bag of words: count how many times each word appears in each document. The result is a matrix where rows are documents, columns are unique words (the vocabulary), and values are counts.

The bag of words discards word order. "The dog bit the man" and "The man bit the dog" have the same representation. This is a severe limitation, but it is sufficient for many tasks, including topic modelling and document classification.

THEORY: TF-IDF derivation

Term Frequency-Inverse Document Frequency weights each word by how important it is to a document, relative to the entire corpus.

Term Frequency (TF): how often word t appears in document d:

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

where $f_{t,d}$ is the raw count of t in d, normalised by the total number of words in d.

Inverse Document Frequency (IDF): how rare the word is across the corpus:

$$\text{idf}(t) = \log \frac{N}{\text{df}(t)}$$

where N is the total number of documents and $\text{df}(t)$ is the number of documents containing t.

TF-IDF is the product:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

Why does this work? A word that appears frequently in a document (high TF) is important to that document. But if that word also appears in every document (low IDF — e.g., "the", "is", "and"), it carries no discriminative

information. The IDF term down-weights common words and up-weights rare, document-specific words.

THEORY: BM25

BM25 (Best Matching 25) is an improved version of TF-IDF used in information retrieval. It introduces two refinements:

Term frequency saturation. In TF-IDF, doubling the word count doubles the score. In BM25, the term frequency component saturates:

$$\text{BM25}(t, d) = \text{idf}(t) \times \frac{f_{t,d} \times (k_1 + 1)}{f_{t,d} + k_1 \times (1 - b + b \times \frac{|d|}{|d_{\text{avg}}|})}$$

where k_1 (typically 1.2–2.0) controls saturation and b (typically 0.75) controls document-length normalisation. The intuition: a word appearing 10 times versus 5 times in a document is not twice as important — there are diminishing returns. And longer documents are expected to have higher counts simply because they have more words, so the score is normalised by document length relative to the average.

BM25 will appear again in Module 6, Lesson 6.4, as the sparse retrieval component of RAG systems.

FOUNDATIONS: Word embeddings (tools, not derivation)

Word embeddings represent each word as a dense vector in a continuous space, where semantically similar words are close together. Three major approaches:

- **Word2Vec (CBOW and Skip-gram):** learns embeddings by predicting a word from its context (CBOW) or context from a word (Skip-gram). The key insight: "words that appear in similar contexts have similar meanings."
- **GloVe:** learns embeddings from global word co-occurrence statistics.
- **FastText:** extends Word2Vec with subword embeddings (character n-grams), so it can handle out-of-vocabulary words.

We use these embeddings as features — we do not yet derive how they are trained. That derivation comes in Lesson 4.8, where you will see that Word2Vec is a shallow neural network, and the embedding vectors are its hidden layer weights. The connection to the Feature Engineering Spectrum: embeddings are features discovered by optimisation, not by hand.

THEORY: LDA — Latent Dirichlet Allocation

LDA is a generative probabilistic model for topic discovery. It assumes each document is a mixture of topics, and each topic is a distribution over words:

$$p(\text{word} \mid \text{document}) = \sum_{k=1}^{K} p(\text{word} \mid \text{topic}_k) \times p(\text{topic}_k \mid \text{document})$$

The generative process for a document:

1. Choose a topic distribution $\theta_d \sim \text{Dirichlet}(\alpha)$.
2. For each word position in the document:
    a. Choose a topic $z \sim \text{Multinomial}(\theta_d)$.
    b. Choose a word $w \sim \text{Multinomial}(\phi_z)$.

The model is fitted using variational inference or collapsed Gibbs sampling — both are variants of the EM framework from Lesson 4.2.

FOUNDATIONS: BERTopic

BERTopic is a modern topic modelling approach that combines:

1. **Sentence embeddings** (from a pre-trained transformer like BERT) to represent documents as dense vectors.
2. **UMAP** (from Lesson 4.3) to reduce dimensionality.
3. **HDBSCAN** (from Lesson 4.1) to cluster the reduced embeddings.
4. **c-TF-IDF** (class-based TF-IDF) to extract topic labels from each cluster.

BERTopic typically produces more coherent and interpretable topics than LDA because it leverages pre-trained language understanding rather than starting from raw word counts.

THEORY: Topic coherence — NPMI

Normalised Pointwise Mutual Information (NPMI) measures how often the top words in a topic co-occur in the corpus:

$$\text{NPMI}(w_i, w_j) = \frac{\log \frac{p(w_i, w_j)}{p(w_i) \cdot p(w_j)}}{-\log p(w_i, w_j)}$$

NPMI ranges from −1 (words never co-occur) to +1 (words always co-occur). A topic's coherence is the average NPMI across all pairs of its top words. Higher coherence means the topic's words genuinely belong together. Coherence scores of 0.05–0.15 are typical for LDA; BERTopic often achieves 0.15–0.25.

The Kailash Engine: AutoMLEngine (NLP mode)

```
from kailash_ml import AutoMLEngine

engine = AutoMLEngine(task="topic_modelling")
topics = engine.extract_topics(documents_df, n_topics=10)
```

Worked Example: Singapore Policy Document Topic Extraction

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import NMF, LatentDirichletAllocation
from bertopic import BERTopic

loader = MLFPDataLoader()
df = loader.load("mlfp04", "sg_policy_submissions.csv")
documents = df["text"].to_list()

# TF-IDF
vectorizer = TfidfVectorizer(max_features=5000, stop_words="english")
tfidf_matrix = vectorizer.fit_transform(documents)
feature_names = vectorizer.get_feature_names_out()

# Method 1: NMF (Non-negative Matrix Factorisation)
nmf = NMF(n_components=8, random_state=42)
W = nmf.fit_transform(tfidf_matrix)  # document-topic matrix
H = nmf.components_                   # topic-word matrix

for i, topic in enumerate(H):
    top_words = [feature_names[j] for j in topic.argsort()[-8:][::-1]]
    print(f"NMF Topic {i}: {', '.join(top_words)}")
```

```
# Method 2: LDA
lda = LatentDirichletAllocation(n_components=8, random_state=42)
lda_topics = lda.fit_transform(tfidf_matrix)

for i, topic in enumerate(lda.components_):
    top_words = [feature_names[j] for j in topic.argsort()[-8:][::-1]]
    print(f"LDA Topic {i}: {', '.join(top_words)}")

# Method 3: BERTopic
topic_model = BERTopic(nr_topics=8)
topics, probs = topic_model.fit_transform(documents)
topic_model.get_topic_info()
```

Try It Yourself

Drill 1. Implement TF-IDF from scratch. Compute the TF and IDF components separately for a small corpus of 5 documents, then multiply them. Verify your result matches `sklearn.feature_extraction.text.TfidfVectorizer`.

Solution:

```
import numpy as np
from collections import Counter

corpus = [
    "singapore housing policy affordable homes",
    "affordable housing development singapore plan",
    "green building construction sustainability",
    "housing affordability young families singapore",
    "sustainable urban development green spaces",
]

# Compute TF
vocab = sorted(set(word for doc in corpus for word in doc.split()))
tf_matrix = np.zeros((len(corpus), len(vocab)))
for i, doc in enumerate(corpus):
    counts = Counter(doc.split())
    total = sum(counts.values())
    for j, word in enumerate(vocab):
        tf_matrix[i, j] = counts.get(word, 0) / total

# Compute IDF
df_counts = np.sum(tf_matrix > 0, axis=0)
idf = np.log(len(corpus) / df_counts)

# TF-IDF
tfidf_manual = tf_matrix * idf
print(f"Shape: {tfidf_manual.shape}")
```

Drill 2. Compare NMF and LDA on the policy documents. Use NPMI coherence to determine which method produces more coherent topics. Print the top 5 words for each topic from both methods.

Solution:

```
from gensim.models.coherencemodel import CoherenceModel
import gensim.corpora as corpora

texts = [doc.split() for doc in documents]
dictionary = corpora.Dictionary(texts)
```

```
for model_name, components in [("NMF", nmf.components_), ("LDA", lda.components_)]:
    topics_words = []
    for topic in components:
        top_idx = topic.argsort()[-10:][::-1]
        topics_words.append([feature_names[j] for j in top_idx])
    cm = CoherenceModel(topics=topics_words, texts=texts, dictionary=dictionary,
        coherence="c_npmi")
    print(f"{model_name} NPMI coherence: {cm.get_coherence():.4f}")
```

Drill 3. Vary the number of LDA topics from 3 to 15 and plot NPMI coherence versus number of topics. What is the optimal number of topics?

Solution:

```
coherences = {}
for n in range(3, 16):
    lda_n = LatentDirichletAllocation(n_components=n, random_state=42)
    lda_n.fit(tfidf_matrix)
    topics_words = []
    for topic in lda_n.components_:
        top_idx = topic.argsort()[-10:][::-1]
        topics_words.append([feature_names[j] for j in top_idx])
    cm = CoherenceModel(topics=topics_words, texts=texts, dictionary=dictionary,
        coherence="c_npmi")
    coherences[n] = cm.get_coherence()
    print(f"n_topics={n}: NPMI={coherences[n]:.4f}")
```

Drill 4. Apply BERTopic to the policy documents and compare with LDA. Which produces more interpretable topic labels? Compute the percentage of documents assigned to each BERTopic topic.

Solution:

```
topic_model = BERTopic(nr_topics=8)
topics, probs = topic_model.fit_transform(documents)
info = topic_model.get_topic_info()
print(info[["Topic", "Count", "Name"]])
```

Drill 5. Implement a simple sentiment classifier using TF-IDF features and logistic regression. Split the policy documents into positive (supportive) and negative (critical) submissions using a labelled subset. Report accuracy and the most predictive words for each sentiment.

Solution:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Assume df has a "sentiment" column (1=positive, 0=negative)
X_train, X_test, y_train, y_test = train_test_split(
    tfidf_matrix, df["sentiment"].to_numpy(), test_size=0.2, random_state=42
)
clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)
acc = clf.score(X_test, y_test)
print(f"Accuracy: {acc:.3f}")

# Most predictive words
for label, name in [(1, "Positive"), (0, "Negative")]:
    if label == 1:
```

```
        top_idx = clf.coef_[0].argsort()[-10:][::-1]
    else:
        top_idx = clf.coef_[0].argsort()[:10]
    words = [feature_names[j] for j in top_idx]
    print(f"{name}: {', '.join(words)}")
```

Cross-References

- **Lesson 4.3** introduced dimensionality reduction. TF-IDF produces a very high-dimensional sparse representation; NMF and LDA reduce it to a low-dimensional dense topic space — the same idea as PCA, applied to text.
- **Lesson 4.7** introduces collaborative filtering, where the user-item matrix plays the same role as the document-term matrix. NMF on text is the same algorithm as NMF on user-item interactions.
- **Lesson 4.8** will explain how Word2Vec learns embeddings. The embeddings you used as tools in this lesson are features discovered by a shallow neural network.
- **Module 6, Lesson 6.4** uses BM25 as the sparse retrieval component of RAG systems.

Reflection

You should now be able to:

- Derive TF-IDF from first principles and explain why IDF down-weights common words.
- Explain BM25's term-frequency saturation and document-length normalisation.
- Distinguish LDA (probabilistic, generative) from BERTopic (embedding-based, discriminative) and explain when each is preferred.
- Evaluate topic quality using NPMI coherence.
- Use word embeddings as features even though you cannot yet explain how they are trained (that comes in Lesson 4.8).

Lesson 4.7: Recommender Systems and Collaborative Filtering

Why This Matters

Shopee, Lazada, and Grab all serve Singapore customers personalised recommendations. When you open Grab Food, the restaurants at the top are not random — they are selected by a recommender system that uses your past orders, the orders of similar users, and the attributes of restaurants to predict what you are most likely to order next. Netflix reported that 80% of the shows people watch are discovered through recommendations, not search. The recommendation algorithm is, for many platforms, the product.

In this lesson you will build three types of recommender systems — content-based, collaborative filtering, and matrix factorisation — and discover the concept that is the intellectual centrepiece of the entire programme: **optimisation drives feature discovery**. Matrix factorisation learns user and item embeddings by minimising reconstruction error. Those embeddings are dense vector representations that capture latent preferences. They are features, but nobody designed them. They emerged from the loss function.

This is THE PIVOT in the Feature Engineering Spectrum. Everything before this lesson discovered features without an error signal (clustering, PCA, topic models). Everything after this lesson discovers features with an error signal (neural networks, deep learning). Matrix factorisation sits at the transition point: it uses optimisation

(like supervised learning) to discover latent structure (like unsupervised learning). Understanding this bridge is understanding the rest of the programme.

Core Concepts

FOUNDATIONS: Three approaches to recommendation

Content-based filtering recommends items similar to what the user has liked before. If you watched three action movies, it recommends more action movies. It uses item features (genre, director, cast) and user preferences (ratings, watch history). Strength: does not need other users' data. Weakness: limited to items similar to what the user has already seen — no surprise.

User-based collaborative filtering finds users similar to you and recommends what they liked. If User A and User B both rated the same five movies highly, and User B liked a sixth movie that User A has not seen, recommend that movie to User A. Strength: can recommend surprising items. Weakness: cold start — new users have no history to compare.

Item-based collaborative filtering finds items similar to the item the user liked. "Customers who bought this also bought that." It computes similarity between items based on who interacted with them. Strength: more stable than user-based (item similarities change slowly). Weakness: same cold-start problem for new items.

THEORY: Matrix factorisation

Consider a user-item interaction matrix $\mathbf{R}$ of dimensions $m \times n$ (users × items), where R_{ui} is user u's rating of item i. Most entries are missing — users have interacted with only a tiny fraction of all items.

Matrix factorisation approximates $\mathbf{R}$ as the product of two low-rank matrices:

$$\mathbf{R} \approx \mathbf{U}\mathbf{V}^T$$

where $\mathbf{U}$ is $m \times k$ (user embeddings) and $\mathbf{V}$ is $n \times k$ (item embeddings), with $k \ll \min(m, n)$.

The predicted rating is:

$$\hat{R}_{ui} = \mathbf{u}_u^T \mathbf{v}_i = \sum_{f=1}^{k} U_{uf} V_{if}$$

The loss function to minimise is:

$$\mathcal{L} = \sum_{(u,i) \in \text{observed}} (R_{ui} - \mathbf{u}_u{}^T \mathbf{v}_i)^2 + \lambda(\|\mathbf{u}_u\|^2 + \|\mathbf{v}_i\|^2)$$

The regularisation term $\lambda(\|\mathbf{u}_u\|^2 + \|\mathbf{v}_i\|^2)$ prevents overfitting to observed ratings.

THEORY: ALS — Alternating Least Squares

The loss function is not jointly convex in $\mathbf{U}$ and $\mathbf{V}$ (it is a product of unknowns). But if you fix $\mathbf{V}$, the problem becomes a standard regularised least squares problem in $\mathbf{U}$ — and vice versa. ALS alternates:

1. Fix $\mathbf{V}$, solve for $\mathbf{U}$: for each user u, $\mathbf{u}_u = (\mathbf{V}_u{}^T \mathbf{V}_u + \lambda \mathbf{I})^{-1} \mathbf{V}_u{}^T \mathbf{r}_u$
2. Fix $\mathbf{U}$, solve for $\mathbf{V}$: for each item i, $\mathbf{v}_i = (\mathbf{U}_i{}^T \mathbf{U}_i + \lambda \mathbf{I})^{-1} \mathbf{U}_i{}^T \mathbf{r}_i$

where $\mathbf{V}_u$ contains only the rows of $\mathbf{V}$ for items rated by user u, and $\mathbf{r}_u$ is the vector of user u's observed ratings.

Each step has a closed-form solution (matrix inversion), so ALS is simple to implement and parallelisable. Convergence is guaranteed because each step decreases (or maintains) the loss.

FOUNDATIONS: Connection to PCA

Recall from Lesson 4.3 that PCA factorises $\mathbf{X} = \mathbf{U\Sigma V}^T$ via SVD. Collaborative filtering factorises $\mathbf{R} \approx \mathbf{UV}^T$. The difference: PCA observes the full matrix; collaborative filtering observes only a sparse subset. Both discover low-rank structure. Both learn embeddings (latent factors). The mechanism is the same: minimise a reconstruction error.

FOUNDATIONS: THE PIVOT — optimisation drives feature discovery

In Lessons 4.1–4.6, unsupervised methods discovered features from the data's geometry — no loss function guided the discovery. Clustering found groups by distance. PCA found directions by variance. LDA found topics by co-occurrence.

In matrix factorisation, features (embeddings) are discovered by minimising a loss function. The user embedding $\mathbf{u}_u$ captures user u's latent preferences. The item embedding $\mathbf{v}_i$ captures item i's latent attributes. Nobody designed these features. They emerged from the optimisation process.

In Lesson 4.8, neural networks will generalise this. A hidden layer's activations are embeddings. They are discovered by minimising a loss function via backpropagation. The difference from matrix factorisation: neural networks can learn non-linear combinations through activation functions, not just linear ones.

The spectrum:

Stage	Method	Features	Error signal?
M3	Manual	Human-designed	N/A
M4.1–4.6	USML	Data geometry	No
M4.7	Matrix factorisation	Optimisation	Yes (reconstruction)
M4.8	Neural networks	Backpropagation	Yes (task loss)
M5	Deep learning	Specialised architectures	Yes (task loss)

Mathematical Foundations

THEORY: Deriving the ALS update for U

Fix $\mathbf{V}$. For user u, the loss restricted to that user's observed ratings is:

$$\mathcal{L}_u = \sum_{i \in \text{rated}_u} (R_{ui} - \mathbf{u}_u{}^T \mathbf{v}_i)^2 + \lambda \|\mathbf{u}_u\|^2$$

Let $\mathbf{V}_u$ be the matrix whose rows are $\mathbf{v}_i{}^T$ for items rated by user u, and $\mathbf{r}_u$ be the vector of observed ratings. Then:

$$\mathcal{L}_u = \|\mathbf{r}_u - \mathbf{V}_u \mathbf{u}_u\|^2 + \lambda \mathbf{u}_u{}^T \mathbf{u}_u$$

Take the derivative with respect to $\mathbf{u}_u$ and set to zero:

$$\nabla_{\mathbf{u}_u} \mathcal{L}_u = -2\mathbf{V}_u{}^T (\mathbf{r}_u - \mathbf{V}_u \mathbf{u}_u) + 2\lambda \mathbf{u}_u = \mathbf{0}$$

$$(\mathbf{V}_u{}^T\mathbf{V}_u + \lambda\mathbf{I})\mathbf{u}_u = \mathbf{V}_u{}^T\mathbf{r}_u$$

$$\mathbf{u}_u = (\mathbf{V}_u{}^T\mathbf{V}_u + \lambda\mathbf{I})^{-1}\mathbf{V}_u{}^T\mathbf{r}_u$$

This is a regularised normal equation — the same structure as ridge regression from Module 2, but applied to learning embeddings instead of regression coefficients.

The Kailash Engine: AutoMLEngine (recommender mode)

```
from kailash_ml import AutoMLEngine

engine = AutoMLEngine(task="recommendation")
result = engine.fit(interactions_df, method="als", factors=50, regularization=0.1)
recommendations = result.recommend(user_id=42, n=10)
```

Worked Example: Singapore Retail Recommender

```
import numpy as np
from scipy.sparse import csr_matrix
from sklearn.metrics.pairwise import cosine_similarity

loader = MLFPDataLoader()
df = loader.load("mlfp04", "sg_retail_interactions.csv")

# Build user-item matrix
users = df["user_id"].unique().sort().to_list()
items = df["item_id"].unique().sort().to_list()
user_idx = {u: i for i, u in enumerate(users)}
item_idx = {it: i for i, it in enumerate(items)}

rows = df["user_id"].map_elements(lambda u: user_idx[u], return_dtype=pl.Int64).to_numpy()
cols = df["item_id"].map_elements(lambda i: item_idx[i], return_dtype=pl.Int64).to_numpy()
vals = df["rating"].to_numpy().astype(float)

R = csr_matrix((vals, (rows, cols)), shape=(len(users), len(items)))

# Method 1: User-based CF
user_sim = cosine_similarity(R)
def recommend_user_cf(user_id, n=5):
    u = user_idx[user_id]
    sim_scores = user_sim[u]
    weighted_ratings = sim_scores @ R.toarray() / (np.abs(sim_scores).sum() + 1e-8)
    already_rated = R[u].toarray().flatten() > 0
    weighted_ratings[already_rated] = -np.inf
    top_items = np.argsort(weighted_ratings)[-n:][::-1]
    return [items[i] for i in top_items]

# Method 2: Item-based CF
item_sim = cosine_similarity(R.T)

# Method 3: ALS Matrix Factorisation
k = 20  # latent factors
lam = 0.1
U = np.random.randn(len(users), k) * 0.01
V = np.random.randn(len(items), k) * 0.01
R_dense = R.toarray()
mask = R_dense > 0
```

```
for iteration in range(20):
    # Update U
    for u in range(len(users)):
        rated = mask[u]
        if rated.sum() == 0:
            continue
        V_u = V[rated]
        r_u = R_dense[u, rated]
        U[u] = np.linalg.solve(V_u.T @ V_u + lam * np.eye(k), V_u.T @ r_u)

    # Update V
    for i in range(len(items)):
        raters = mask[:, i]
        if raters.sum() == 0:
            continue
        U_i = U[raters]
        r_i = R_dense[raters, i]
        V[i] = np.linalg.solve(U_i.T @ U_i + lam * np.eye(k), U_i.T @ r_i)

    # Compute loss
    pred = U @ V.T
    loss = np.sum((R_dense[mask] - pred[mask])**2) + lam * (np.sum(U**2) + np.sum(V**2))
    if iteration % 5 == 0:
        print(f"Iteration {iteration}: loss = {loss:.2f}")

# Visualise embeddings (project to 2D with PCA)
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
V_2d = pca.fit_transform(V)
# Items that are close in embedding space are similar
```

Try It Yourself

Drill 1. Implement user-based collaborative filtering with cosine similarity. Recommend 5 items for 3 different users. For each recommendation, explain which similar user's preferences drove the recommendation.

Solution:

```
for uid in [users[0], users[10], users[50]]:
    recs = recommend_user_cf(uid, n=5)
    u = user_idx[uid]
    most_similar = np.argsort(user_sim[u])[-2]  # -1 is self
    print(f"User {uid}: recommended {recs}")
    print(f"  Most similar user: {users[most_similar]}")
```

Drill 2. Implement item-based collaborative filtering. For a given user who rated item A highly, find the 5 most similar items to A and recommend them. Compare with user-based CF recommendations.

Solution:

```
def recommend_item_cf(user_id, n=5):
    u = user_idx[user_id]
    user_ratings = R[u].toarray().flatten()
    rated_items = np.where(user_ratings > 0)[0]
    scores = np.zeros(len(items))
    for i in rated_items:
        scores += user_ratings[i] * item_sim[i]
```

```
    scores[rated_items] = -np.inf
    top_items = np.argsort(scores)[-n:][::-1]
    return [items[i] for i in top_items]
```

Drill 3. Vary the number of latent factors k in ALS from 5 to 100 (5, 10, 20, 50, 100). Plot the reconstruction error versus k. What is the optimal k based on a validation set?

Solution:

```
for k in [5, 10, 20, 50, 100]:
    # Run ALS with k factors
    # Split observed ratings into 80% train, 20% validation
    # Report train and validation RMSE
    print(f"k={k}: train_rmse=..., val_rmse=...")
```

Drill 4. Visualise the item embeddings from the ALS model in 2D (using PCA or UMAP). Colour the items by category. Do items in the same category cluster together in embedding space?

Solution:

```
import umap
reducer = umap.UMAP(n_components=2, random_state=42)
V_2d = reducer.fit_transform(V)
# Plot with category colours
```

Drill 5. Write a paragraph explaining the pivot concept: how does matrix factorisation bridge unsupervised feature discovery (Lessons 4.1–4.6) and supervised feature learning (Lesson 4.8 and Module 5)? Use the terms "embedding", "loss function", and "optimisation" in your explanation.

Solution: Matrix factorisation learns embeddings — dense vector representations of users and items — by minimising a reconstruction loss function through optimisation (ALS). Unlike unsupervised methods like PCA or clustering, which discover structure from data geometry alone, matrix factorisation uses an error signal (the gap between predicted and observed ratings) to guide feature discovery. This is exactly what neural networks do: their hidden layer activations are embeddings, discovered by minimising a task loss via backpropagation. The difference is that matrix factorisation learns linear combinations, while neural networks learn non-linear combinations through activation functions.

Cross-References

- **Lesson 4.3** derived PCA via SVD. Matrix factorisation is SVD applied to a sparse matrix — the same mathematical operation on different data.
- **Lesson 4.6** used NMF for topic modelling. NMF on a document-term matrix and NMF on a user-item matrix are the same algorithm.
- **Lesson 4.8** will generalise the idea: neural network hidden layers are embeddings learned by minimising a loss function, with the addition of non-linearity.
- **Module 5, Lesson 5.1** introduces autoencoders, which learn embeddings by reconstructing their input — the same objective as matrix factorisation, but with a neural network.

Reflection

You should now be able to:

- Build content-based, user-based CF, and item-based CF recommenders.
- Implement ALS matrix factorisation from scratch and explain why it converges.
- Derive the ALS update rule as a regularised normal equation.

- Explain the pivot concept: optimisation drives feature discovery.
- Visualise learned embeddings and interpret what they capture.
- Articulate the connection between matrix factorisation and PCA.

Lesson 4.8: Neural Networks, Backpropagation, and the DL Training Toolkit

Why This Matters

This is the most important lesson in the MLFP programme. Not because neural networks are the most important algorithm (they are not — gradient boosting still wins most tabular competitions). But because this lesson completes the bridge from classical ML to deep learning, and everything in Modules 5 and 6 builds on it.

In Lesson 4.7 you saw that matrix factorisation learns embeddings by minimising a reconstruction loss. Those embeddings are linear combinations of the input. But real-world patterns are rarely linear. The relationship between a customer's purchase history and their next purchase involves interactions, thresholds, and non-linear effects that no linear model can capture.

A neural network with hidden layers learns non-linear combinations. Each hidden layer applies a linear transformation (multiply by weights, add bias) followed by a non-linear activation function. The activations of the hidden layers are the embeddings — features discovered by the network. The key insight: **hidden layers are automated feature engineering with error feedback**. The network writes its own features, guided by the loss function, through backpropagation.

This lesson covers the complete DL training toolkit: forward pass, backpropagation, gradient descent, activation functions, optimisers, loss functions, dropout, batch normalisation, weight initialisation, learning rate schedules, and gradient clipping. By the end, you will be able to build, train, and diagnose a neural network from scratch.

Core Concepts

FOUNDATIONS: From linear regression to neural networks

Linear regression predicts $\hat{y} = \mathbf{w}^T\mathbf{x} + b$. This is a single-layer network with no activation function. It can only learn linear relationships.

Add a hidden layer with an activation function:

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\hat{y} = \mathbf{w}_2{}^T\mathbf{h} + b_2$$

where σ is a non-linear activation function (like ReLU). The hidden layer $\mathbf{h}$ is a new set of features — not designed by a human, but learned from the data. Adding more hidden layers allows the network to compose non-linear transformations, learning increasingly abstract features.

The Universal Approximation Theorem states that a neural network with a single hidden layer of sufficient width can approximate any continuous function on a compact set to arbitrary precision. In practice, deeper networks (more layers) tend to learn more efficiently than very wide shallow networks.

THEORY: The forward pass

For a network with L layers:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \text{(linear transformation)}$$
$$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}) \quad \text{(activation function)}$$

where $\mathbf{a}^{(0)} = \mathbf{x}$ (the input), $\mathbf{W}^{(l)}$ is the weight matrix for layer l, $\mathbf{b}^{(l)}$ is the bias vector, and $f^{(l)}$ is the activation function.

The final output $\hat{y} = \mathbf{a}^{(L)}$ is compared to the true value y using a loss function $\mathcal{L}(y, \hat{y})$.

THEORY: Backpropagation — the chain rule through layers

Backpropagation computes the gradient of the loss with respect to every weight in the network, using the chain rule of calculus. For a single weight $W_{jk}^{(l)}$ in layer l:

$$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \cdot \frac{\partial \mathbf{a}^{(L)}}{\partial \mathbf{z}^{(L)}} \cdot \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{a}^{(L-1)}} \cdots \frac{\partial \mathbf{z}^{(l)}}{\partial W_{jk}^{(l)}}$$

Define the error signal (delta) for layer l:

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$$

For the output layer: $\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}}\mathcal{L} \odot f'^{(L)}(\mathbf{z}^{(L)})$

For hidden layers (propagating backward): $\boldsymbol{\delta}^{(l)} = (\mathbf{W}^{(l+1)T}\boldsymbol{\delta}^{(l+1)}) \odot f'^{(l)}(\mathbf{z}^{(l)})$

The gradient with respect to the weights: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^T$

The gradient with respect to the biases: $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$

THEORY: Gradient descent

Update each weight in the direction that decreases the loss:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$$

where η is the learning rate. Too large: overshoots and diverges. Too small: converges too slowly. The learning rate is the single most important hyperparameter in neural network training.

FOUNDATIONS: Activation functions

Function	Formula	Use	Why
ReLU	$\max(0, z)$	Default hidden layer	Simple, fast, mitigates vanishing gradient
Leaky ReLU	$\max(0.01z, z)$	Hidden layer	Avoids dead neurons

Function	Formula	Use	Why
GELU	$z \cdot \Phi(z)$	Transformer hidden layers	Smooth, used in BERT/GPT
Sigmoid	$1/(1 + e^{-z})$	Binary output	Maps to $[0, 1]$ probability
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	Hidden layer (less common)	Zero-centred, maps to $[-1, 1]$
Softmax	$e^{z_i}/\sum_j e^{z_j}$	Multi-class output	Maps to probability distribution

ReLU is the default choice for hidden layers. Sigmoid and softmax are for output layers. GELU is the default in modern transformer architectures.

THEORY: Loss functions taxonomy

Loss	Formula	Use
MSE	$\frac{1}{n}\sum(y - \hat{y})^2$	Regression
MAE	$\frac{1}{n}\sum\|y - \hat{y}\|$	Robust regression
Cross-entropy	$-\sum y_c \log \hat{y}_c$	Multi-class classification
Binary CE	$-[y \log \hat{y} + (1 - y) \log (1 - \hat{y})]$	Binary classification
Focal loss	$-\alpha_t (1 - p_t)^\gamma \log (p_t)$	Imbalanced classification
KL divergence	$\sum p \log (p/q)$	Distribution matching (VAE)

FOUNDATIONS: Dropout

Dropout randomly sets a fraction p of the hidden layer activations to zero during training. This forces the network to learn redundant representations — no single neuron can be relied upon, so the knowledge must be distributed. During inference, dropout is turned off and activations are scaled by $(1 - p)$ to compensate.

Dropout rate is typically 0.1–0.5. Higher rates provide stronger regularisation but slow convergence. It is the neural network equivalent of bagging — each training step uses a different random subset of neurons, effectively training an ensemble of networks.

THEORY: Batch normalisation

Batch normalisation normalises the inputs to each layer to have zero mean and unit variance within each mini-batch:

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{z}_i + \beta$$

where μ_B and ${\sigma_B}^2$ are the mini-batch mean and variance, γ and β are learned scale and shift parameters, and ϵ is a small constant for numerical stability.

Benefits: stabilises training (layers receive inputs with consistent statistics), enables higher learning rates, acts as a mild regulariser.

FOUNDATIONS: Weight initialisation

If all weights are initialised to zero, all neurons compute the same output, all gradients are the same, and the network never breaks symmetry — it cannot learn. Random initialisation breaks symmetry, but the scale matters:

- **Xavier/Glorot** (for sigmoid/tanh): $W \sim \mathcal{N}(0, 2/(n_{\text{in}} + n_{\text{out}}))$
- **Kaiming/He** (for ReLU): $W \sim \mathcal{N}(0, 2/n_{\text{in}})$

Kaiming initialisation accounts for the fact that ReLU zeros out half the activations, so the variance must be doubled to compensate.

THEORY: Optimisers

SGD with momentum maintains a running average of gradients:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla \mathcal{L}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \mathbf{v}_t$$

Adam (Adaptive Moment Estimation) adapts the learning rate for each parameter:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t)$$

$$W \leftarrow W - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

Adam is the default optimiser for most deep learning tasks. AdamW adds decoupled weight decay, which is preferred for transformer training.

FOUNDATIONS: Learning rate schedules

A fixed learning rate is rarely optimal. Common schedules:

- **Step decay:** reduce by a factor every N epochs.
- **Cosine annealing:** $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t / T))$
- **Warmup + cosine:** start with a low learning rate, linearly increase to the peak, then follow cosine decay. Used in transformer training.
- **ReduceLROnPlateau:** reduce when validation loss stops improving.

FOUNDATIONS: Gradient clipping and early stopping

Gradient clipping prevents exploding gradients by capping the gradient norm:

$$\text{if } \|\nabla \mathcal{L}\| > \text{max_norm}: \quad \nabla \mathcal{L} \leftarrow \text{max_norm} \cdot \frac{\nabla \mathcal{L}}{\|\nabla \mathcal{L}\|}$$

Essential for RNNs (Module 5) and transformers where gradients can grow explosively.

Early stopping monitors validation loss and stops training when it begins to increase (patience of N epochs). This prevents overfitting — the model's performance on unseen data degrades even as training loss continues to decrease.

Mathematical Foundations

THEORY: Backpropagation derivation for a 2-layer network

Consider a network with one hidden layer:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)})$$

$$\hat{y} = \mathbf{w}^{(2)T}\mathbf{a}^{(1)} + b^{(2)}$$

$$\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$$

Output layer gradients:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -(y - \hat{y})$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \mathbf{a}^{(1)} = -(y - \hat{y}) \mathbf{a}^{(1)}$$

Hidden layer gradients (chain rule):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \mathbf{w}^{(2)} = -(y - \hat{y}) \mathbf{w}^{(2)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(1)}} \odot \text{ReLU}'(\mathbf{z}^{(1)})$$

where $\text{ReLU}'(z) = \mathbf{1}[z > 0]$ (1 if positive, 0 otherwise).

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} \mathbf{x}^T$$

This is backpropagation: compute the error at the output, propagate it backward through each layer using the chain rule, and use the result to compute the gradient for each weight.

ADVANCED: Hidden layers as automated feature engineering

Consider a 2-hidden-layer network for HDB price prediction. The input is $\mathbf{x}$ = [floor_area, storey, lease_remaining, town_encoded].

The first hidden layer might learn features like: - h_1: “overall quality” (positive loading on area, storey, and lease) - h_2: “location premium” (depends heavily on town encoding) - h_3: “new vs old” (positive on lease remaining, negative on storey)

The second hidden layer combines these into more abstract features: - h_1': “premium mature estate flat” (combines location premium with overall quality) - h_2': “value new-build” (combines new-vs-old with moderate quality)

These features were not designed by anyone. They emerged from minimising the price prediction error via backpropagation. This is representation learning — the network discovers its own representations.

The connection to Module 4’s journey: in Lesson 4.3, PCA found linear combinations that maximise variance. In Lesson 4.7, matrix factorisation found linear combinations that minimise reconstruction error. Here, neural networks find non-linear combinations that minimise task-specific loss. Each step adds more power.

The Kailash Engine: OnnxBridge (model export)

```
from kailash_ml import OnnxBridge

bridge = OnnxBridge()
bridge.export(model, input_shape=(1, 4), output_path="hdb_predictor.onnx")
# Load for inference
loaded = bridge.load("hdb_predictor.onnx")
prediction = loaded.predict(sample_input)
```

Worked Example: Neural Network for HDB Price Prediction — from Scratch

We build a 3-layer network from scratch using NumPy, then add each training technique one by one to see its effect.

```
import numpy as np

# Load HDB data
loader = MLFPDataLoader()
df = loader.load("mlfp04", "sg_hdb_prices.csv")
features = ["floor_area_sqm", "storey_range_mid", "remaining_lease_years", "town_encoded"]
X = df.select(features).to_numpy().astype(np.float64)
y = df["resale_price"].to_numpy().astype(np.float64).reshape(-1, 1)

# Standardise
X_mean, X_std = X.mean(axis=0), X.std(axis=0)
y_mean, y_std = y.mean(), y.std()
X_norm = (X - X_mean) / X_std
y_norm = (y - y_mean) / y_std

# Split
n_train = int(0.8 * len(X_norm))
X_train, X_test = X_norm[:n_train], X_norm[n_train:]
y_train, y_test = y_norm[:n_train], y_norm[n_train:]

# Network architecture: 4 -> 64 -> 32 -> 1
np.random.seed(42)
W1 = np.random.randn(4, 64) * np.sqrt(2.0 / 4)   # Kaiming init
b1 = np.zeros((1, 64))
W2 = np.random.randn(64, 32) * np.sqrt(2.0 / 64)
b2 = np.zeros((1, 32))
W3 = np.random.randn(32, 1) * np.sqrt(2.0 / 32)
b3 = np.zeros((1, 1))

def relu(z):
    return np.maximum(0, z)

def relu_grad(z):
    return (z > 0).astype(float)

lr = 0.001
batch_size = 64
epochs = 100

for epoch in range(epochs):
    # Shuffle
    perm = np.random.permutation(n_train)
```

```
    X_shuffled = X_train[perm]
    y_shuffled = y_train[perm]

    epoch_loss = 0
    for start in range(0, n_train, batch_size):
        end = min(start + batch_size, n_train)
        X_batch = X_shuffled[start:end]
        y_batch = y_shuffled[start:end]
        m = len(X_batch)

        # Forward pass
        z1 = X_batch @ W1 + b1
        a1 = relu(z1)
        z2 = a1 @ W2 + b2
        a2 = relu(z2)
        z3 = a2 @ W3 + b3
        y_hat = z3  # linear output for regression

        # Loss (MSE)
        loss = np.mean((y_batch - y_hat) ** 2)
        epoch_loss += loss * m

        # Backward pass
        dz3 = -2 * (y_batch - y_hat) / m
        dW3 = a2.T @ dz3
        db3 = dz3.sum(axis=0, keepdims=True)

        da2 = dz3 @ W3.T
        dz2 = da2 * relu_grad(z2)
        dW2 = a1.T @ dz2
        db2 = dz2.sum(axis=0, keepdims=True)

        da1 = dz2 @ W2.T
        dz1 = da1 * relu_grad(z1)
        dW1 = X_batch.T @ dz1
        db1 = dz1.sum(axis=0, keepdims=True)

        # Update weights
        W3 -= lr * dW3
        b3 -= lr * db3
        W2 -= lr * dW2
        b2 -= lr * db2
        W1 -= lr * dW1
        b1 -= lr * db1

    epoch_loss /= n_train
    if epoch % 20 == 0:
        # Test loss
        z1_t = X_test @ W1 + b1; a1_t = relu(z1_t)
        z2_t = a1_t @ W2 + b2; a2_t = relu(z2_t)
        y_hat_t = a2_t @ W3 + b3
        test_loss = np.mean((y_test - y_hat_t) ** 2)
        print(f"Epoch {epoch}: train_loss={epoch_loss:.4f}, test_loss={test_loss:.4f}")
```

The training loss should decrease steadily. If the test loss begins to increase while training loss continues to decrease, you are overfitting — and that is where dropout, batch norm, and early stopping come in.

Try It Yourself

Drill 1. Add dropout to the hidden layers (p=0.2). Implement it from scratch: during training, generate a binary mask from Bernoulli(1-p) and element-wise multiply the activations. Scale by 1/(1-p). During evaluation, do not apply dropout. Compare training curves with and without dropout.

Solution:

```
def dropout(a, p=0.2, training=True):
    if not training:
        return a
    mask = (np.random.rand(*a.shape) > p).astype(float)
    return a * mask / (1 - p)

# In forward pass during training:
a1 = dropout(relu(z1), p=0.2, training=True)
a2 = dropout(relu(z2), p=0.2, training=True)
```

Drill 2. Implement batch normalisation from scratch for the first hidden layer. During training, normalise using the mini-batch statistics. Maintain running mean and variance for inference. Compare training convergence with and without batch norm.

Solution:

```
gamma1 = np.ones((1, 64))
beta1 = np.zeros((1, 64))
running_mean = np.zeros((1, 64))
running_var = np.ones((1, 64))
momentum = 0.1

def batch_norm(z, gamma, beta, running_mean, running_var, training=True):
    if training:
        mu = z.mean(axis=0, keepdims=True)
        var = z.var(axis=0, keepdims=True)
        running_mean[:] = (1 - momentum) * running_mean + momentum * mu
        running_var[:] = (1 - momentum) * running_var + momentum * var
    else:
        mu = running_mean
        var = running_var
    z_hat = (z - mu) / np.sqrt(var + 1e-8)
    return gamma * z_hat + beta
```

Drill 3. Replace SGD with Adam. Implement Adam from scratch (maintain first and second moment estimates, apply bias correction). Compare convergence speed: how many epochs does SGD need versus Adam to reach the same test loss?

Solution:

```
# Adam state for each parameter
m_W1 = np.zeros_like(W1); v_W1 = np.zeros_like(W1)
t = 0
beta1_adam, beta2_adam, eps_adam = 0.9, 0.999, 1e-8

def adam_update(param, grad, m, v, t, lr=0.001):
    m = beta1_adam * m + (1 - beta1_adam) * grad
    v = beta2_adam * v + (1 - beta2_adam) * grad**2
    m_hat = m / (1 - beta1_adam**t)
    v_hat = v / (1 - beta2_adam**t)
    param -= lr * m_hat / (np.sqrt(v_hat) + eps_adam)
    return param, m, v
```

Drill 4. Implement cosine annealing for the learning rate. Start at $\eta = 0.001$, anneal to $\eta = 0.0001$ over 100 epochs. Plot the learning rate schedule and compare training curves with fixed vs cosine-annealed learning rate.

Solution:

```
eta_max, eta_min = 0.001, 0.0001
T = 100
for epoch in range(T):
    lr = eta_min + 0.5 * (eta_max - eta_min) * (1 + np.cos(np.pi * epoch / T))
    # Use lr for this epoch's updates
```

Drill 5. Implement gradient clipping with max_norm = 1.0. Compute the total gradient norm across all parameters. If it exceeds max_norm, scale all gradients down proportionally. When does gradient clipping activate during training? In which epochs?

Solution:

```
def clip_gradients(grads, max_norm=1.0):
    total_norm = np.sqrt(sum(np.sum(g**2) for g in grads))
    if total_norm > max_norm:
        scale = max_norm / total_norm
        grads = [g * scale for g in grads]
    return grads, total_norm

# After computing all gradients:
[dW1, dW2, dW3, db1, db2, db3], norm = clip_gradients(
    [dW1, dW2, dW3, db1, db2, db3], max_norm=1.0
)
if norm > 1.0:
    print(f"Epoch {epoch}: gradient clipped (norm={norm:.2f})")
```

Drill 6. Extract the activations of the first hidden layer for all test data points. Apply PCA to reduce these 64-dimensional activations to 2D. Colour the scatter plot by the true resale price. Do the learned representations show meaningful structure (e.g., expensive flats clustered together)?

Solution:

```
z1_test = X_test @ W1 + b1
a1_test = relu(z1_test)
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
embeddings_2d = pca.fit_transform(a1_test)
# Plot with colour = y_test (resale price)
```

Cross-References

- **Module 2** introduced gradient descent for linear regression. Neural network training uses the same principle, extended to multiple layers via the chain rule.
- **Lesson 4.3** derived PCA as linear feature extraction. Hidden layer activations are non-linear feature extraction — a generalisation.
- **Lesson 4.7** introduced optimisation-driven feature discovery via matrix factorisation. Neural networks extend this with non-linearity and depth.
- **Module 5** builds on every concept in this lesson: autoencoders (5.1), CNNs (5.2), RNNs (5.3), transformers (5.4), GANs (5.5), GNNs (5.6), transfer learning (5.7), and RL (5.8).

Reflection

You should now be able to:

- Build a neural network from scratch: forward pass, loss, backpropagation, weight update.
- Derive the backpropagation equations for a 2-layer network using the chain rule.
- Explain why hidden layers are automated feature engineering with error feedback.
- Select the appropriate activation function, optimiser, and loss function for a given task.
- Implement dropout and batch normalisation from scratch.
- Apply learning rate scheduling and gradient clipping.
- Articulate the complete Feature Engineering Spectrum from manual features (M3) to learned features (M4.8).

This lesson completes the Module 4 arc. You entered this chapter knowing how to design features by hand. You now understand that machines can design features for themselves, and the mechanism is optimisation. In Module 5 you will see specialised neural architectures that exploit the structure of images (CNNs), sequences (RNNs), graphs (GNNs), and attention (transformers). Each one is a variation on the same theme: learn features from data, guided by a loss function, via backpropagation. The vocabulary you built in this lesson — forward pass, chain rule, gradient descent, dropout, batch norm, Adam — will be on your fingertips for the next 16 lessons.

Chapter Summary

Module 4 took you from the last row of labelled data in Module 3 into the territory of unlabelled data and beyond. The arc has a clear shape.

The first half (Lessons 4.1–4.6) was unsupervised machine learning: discovering structure in data without labels. Clustering found groups. PCA found directions. Topic modelling found themes. Anomaly detection found outliers. Association rules found co-occurrences. In every case, the features were discovered from the data's own geometry — no error signal, no loss function, no gradient.

The pivot (Lesson 4.7) introduced optimisation-driven feature discovery. Matrix factorisation learns embeddings by minimising reconstruction error. The embeddings are features, but nobody designed them — they emerged from the loss function. This is the bridge between unsupervised and supervised feature learning.

The second half (Lesson 4.8) generalised the pivot to neural networks. Hidden layers are automated feature engineering with error feedback. The activations are embeddings, learned by backpropagation. Non-linear activation functions allow the network to learn feature combinations that no linear method can capture.

The Feature Engineering Spectrum — completed

Stage	Module	Method	Features	Error signal
Manual	M3	Domain expertise	Human-designed	N/A
Geometric	M4.1–4.3	Clustering, PCA	Data structure	No
Statistical	M4.4–4.6	Anomaly, topics, rules	Co-occurrence	No
Optimisation	M4.7	Matrix factorisation	Embeddings (linear)	Yes (reconstruction)
Learned	M4.8	Neural networks	Embeddings (non-linear)	Yes (task loss)
Specialised	M5	CNN, RNN, Transformer	Architecture-specific	Yes (task loss)
Semantic	M6	LLMs	Language features	Yes (pre-training)

This spectrum is the intellectual backbone of the MLFP programme. Every module from here forward is a variation on “learn features from data, guided by a loss function”.

What Module 5 builds on

Module 5 assumes you can:

- Implement a neural network from scratch (forward pass, backprop, gradient descent).
- Use dropout, batch normalisation, weight initialisation, and Adam.
- Read training curves and diagnose overfitting, underfitting, and gradient pathologies.
- Export models with OnnxBridge.
- Explain representation learning: hidden layers discover features.

Module 5 introduces specialised architectures: autoencoders for reconstruction, CNNs for spatial data, RNNs for sequential data, transformers for attention-based processing, GANs for generation, GNNs for graph data, transfer learning for reuse, and reinforcement learning for interaction. Each architecture imposes a structural bias that makes learning efficient for a specific data type. The DL training toolkit from Lesson 4.8 — activation functions, optimisers, loss functions, regularisation — applies to all of them.

Glossary

Activation function. A non-linear function applied element-wise to a layer’s output. Introduces non-linearity into the network. Common choices: ReLU, sigmoid, tanh, GELU, softmax.

Adam. Adaptive Moment Estimation. An optimiser that maintains per-parameter learning rates using first and second moment estimates of the gradient.

Agglomerative clustering. A hierarchical clustering method that starts with each point as its own cluster and iteratively merges the two closest clusters.

ALS (Alternating Least Squares). An optimisation algorithm for matrix factorisation that alternates between fixing user embeddings and solving for item embeddings, and vice versa.

Anomaly. A data point that differs significantly from the majority. Also called an outlier.

Apriori algorithm. A frequent itemset mining algorithm that generates candidates bottom-up and prunes using the Apriori principle: infrequent itemsets cannot have frequent supersets.

ARI (Adjusted Rand Index). An external cluster evaluation metric that measures agreement between predicted and true labels, adjusted for chance.

Association rule. A rule of the form $X \rightarrow Y$ discovered from transaction data, scored by support, confidence, and lift.

Autoencoder. A neural network that learns to reconstruct its input through a bottleneck, thereby learning compressed representations.

Backpropagation. The algorithm for computing gradients of the loss with respect to all weights in a neural network, using the chain rule of calculus.

Batch normalisation. A technique that normalises layer inputs to zero mean and unit variance within each mini-batch, stabilising training.

BERTopic. A modern topic modelling approach combining sentence embeddings, UMAP, HDBSCAN, and class-based TF-IDF.

BM25. An information retrieval scoring function that improves on TF-IDF with term frequency saturation and document length normalisation.

Centroid. The mean of all points in a cluster. Used in K-means as the cluster representative.

Cluster. A group of data points that are more similar to each other than to points in other groups.

Collaborative filtering. A recommendation technique that predicts a user's preferences based on the preferences of similar users or items.

Confidence (association rules). The conditional probability $P(Y \mid X)$ — how often Y appears in transactions that contain X.

Content-based filtering. A recommendation technique that uses item features to recommend items similar to those the user has previously liked.

Cosine annealing. A learning rate schedule that follows a cosine curve from maximum to minimum over the training period.

Cosine similarity. A measure of similarity between two vectors based on the cosine of the angle between them: $\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$.

Covariance matrix. A symmetric matrix whose (i,j) entry is the covariance between features i and j. The eigenvalues and eigenvectors of the covariance matrix are the foundation of PCA.

Cross-entropy loss. A loss function for classification that measures the divergence between predicted class probabilities and true labels.

Curse of dimensionality. The phenomenon where high-dimensional spaces become increasingly sparse, making distance-based methods ineffective.

Davies-Bouldin Index. An internal cluster evaluation metric. Lower values indicate better-defined clusters.

DBSCAN. Density-Based Spatial Clustering of Applications with Noise. A clustering algorithm that defines clusters as dense regions separated by sparse regions.

Dendrogram. A tree diagram showing the hierarchy of cluster merges in hierarchical clustering.

Dimensionality reduction. Reducing the number of features while preserving important structure. PCA, t-SNE, and UMAP are dimensionality reduction methods.

Dropout. A regularisation technique that randomly zeros out a fraction of neurons during training, forcing the network to learn distributed representations.

Early stopping. Halting training when validation loss stops improving, to prevent overfitting.

Eigenvalue. A scalar λ such that $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ for some non-zero vector $\mathbf{v}$. In PCA, eigenvalues represent the variance along each principal component.

Eigenvector. A non-zero vector $\mathbf{v}$ such that $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. In PCA, eigenvectors are the principal component directions.

Elbow method. A heuristic for choosing K in K-means by plotting WCSS versus K and identifying the "elbow" where the rate of decrease slows.

EM algorithm. Expectation-Maximisation. An iterative algorithm for fitting latent-variable models. Alternates between computing posterior probabilities of latent variables (E-step) and updating model parameters (M-step).

Embedding. A dense vector representation of a high-dimensional or discrete object (word, user, item) in a continuous low-dimensional space, learned through optimisation.

EnsembleEngine. Kailash ML engine for combining multiple models via blending, stacking, bagging, or boosting.

Feature Engineering Spectrum. The organising framework of the MLFP curriculum: from manual features (M3) through unsupervised discovery (M4.1–4.6) to optimisation-driven learning (M4.7) to neural representation learning (M4.8+).

Forward pass. Computing the output of a neural network by passing input through each layer sequentially.

FP-Growth. A frequent itemset mining algorithm that uses a compressed FP-tree to extract patterns without candidate generation.

Gap statistic. A method for choosing the number of clusters by comparing within-cluster dispersion to a null reference distribution.

Gaussian Mixture Model (GMM). A probabilistic model that represents data as a mixture of Gaussian distributions, fitted using the EM algorithm.

GELU. Gaussian Error Linear Unit. An activation function used in transformer architectures: $\mathrm{GELU}(z) = z \cdot \Phi(z)$.

Gradient clipping. Limiting the magnitude of gradients during training to prevent exploding gradient problems.

Gradient descent. An optimisation algorithm that iteratively adjusts parameters in the direction that decreases the loss function.

HDBSCAN. Hierarchical DBSCAN. A density-based clustering algorithm that automatically selects the density threshold per cluster.

Hidden layer. A layer in a neural network between the input and output layers. Its activations are learned features.

IDF (Inverse Document Frequency). A measure of how rare a word is across a corpus: $\log(N/\mathrm{df}(t))$.

IQR (Interquartile Range). The difference between the 75th and 25th percentiles. Used for outlier detection: values outside $Q_1 - 1.5 \times \mathrm{IQR}$ to $Q_3 + 1.5 \times \mathrm{IQR}$ are flagged.

Isolation Forest. An anomaly detection algorithm that isolates anomalies using random trees. Short path length indicates anomaly.

K-means. A clustering algorithm that partitions data into K clusters by iteratively assigning points to the nearest centroid and recomputing centroids.

K-means++. An initialisation strategy for K-means that spreads initial centroids apart, improving convergence.

Kaiming initialisation. Weight initialisation designed for ReLU networks: $W \sim \mathcal{N}(0, 2/n_{\text{in}})$.

LDA (Latent Dirichlet Allocation). A generative probabilistic model for topic discovery that treats documents as mixtures of topics and topics as distributions over words.

Learning rate. The step size in gradient descent. Controls how much weights are updated per iteration.

Lift (association rules). The ratio of observed co-occurrence to expected co-occurrence under independence. Lift > 1 indicates positive association.

Linkage. The criterion for measuring distance between clusters in hierarchical clustering: single, complete, average, or Ward's.

Loading (PCA). The weight of an original feature in a principal component. Used for interpreting what each component represents.

Local Outlier Factor (LOF). An anomaly detection method that compares a point's local density to the local densities of its neighbours.

Loss function. A function that measures the discrepancy between model predictions and true values. Training minimises the loss.

Matrix factorisation. Decomposing a matrix into the product of two lower-rank matrices. Used in recommender systems and topic modelling.

Mixture of Experts (MoE). A model architecture where a gating network routes inputs to specialised sub-networks (experts).

NMF (Non-negative Matrix Factorisation). A matrix factorisation method that constrains both factor matrices to have non-negative entries. Used for topic modelling and recommender systems.

NMI (Normalised Mutual Information). An external cluster evaluation metric based on information theory.

NPMI (Normalised Pointwise Mutual Information). A coherence metric for topic models that measures word co-occurrence normalised by individual frequencies.

PCA (Principal Component Analysis). A dimensionality reduction method that finds the directions of maximum variance in the data via eigendecomposition of the covariance matrix.

Perplexity (t-SNE). A parameter controlling the effective number of neighbours in t-SNE. Typically 5–50.

Reconstruction error. The difference between the original data and its approximation from a reduced representation.

ReLU. Rectified Linear Unit. $f(z) = \max(0, z)$. The default activation function for hidden layers.

Representation learning. Learning data representations (features) that make downstream tasks easier. Hidden layer activations are learned representations.

Responsibility (EM). The posterior probability that a data point was generated by a particular component, computed in the E-step.

Score blending. Combining normalised scores from multiple detectors using a weighted average to improve anomaly detection robustness.

Scree plot. A plot of eigenvalues (or variance explained) versus component index, used to choose the number of PCA components.

Silhouette score. An internal cluster evaluation metric that measures how similar a point is to its own cluster versus other clusters. Range: $[-1, +1]$.

Singular Value Decomposition (SVD). The factorisation $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal and $\mathbf{\Sigma}$ is diagonal.

Soft clustering. Assigning each point a probability of belonging to each cluster, rather than a hard binary assignment. GMMs produce soft clusters.

Spectral clustering. A clustering method that uses eigenvalues of the graph Laplacian to partition data. Effective for non-convex clusters.

Support (association rules). The fraction of transactions containing an itemset: $\text{supp}(X) = |\{t : X \subseteq t\}|/|T|$.

t-SNE. t-distributed Stochastic Neighbour Embedding. A non-linear dimensionality reduction method for visualisation that preserves local structure.

TF-IDF. Term Frequency-Inverse Document Frequency. A text representation that weights words by their importance to a document relative to the corpus.

Topic model. A model that discovers latent themes (topics) in a collection of documents. LDA and BERTopic are topic models.

UMAP. Uniform Manifold Approximation and Projection. A non-linear dimensionality reduction method that preserves both local and global structure.

Universal Approximation Theorem. The theorem that a sufficiently wide single-hidden-layer neural network can approximate any continuous function on a compact set.

Variance explained. The proportion of total variance captured by a subset of principal components.

Ward's linkage. A hierarchical clustering linkage method that minimises the increase in total within-cluster variance at each merge.

WCSS (Within-Cluster Sum of Squares). The K-means objective function: the sum of squared distances from each point to its cluster centroid.

Weight initialisation. The method for setting initial values of neural network weights. Xavier/Glorot for sigmoid/tanh, Kaiming/He for ReLU.

Xavier initialisation. Weight initialisation designed for sigmoid and tanh networks: $W \sim \mathcal{N}(0, 2/(n_{\text{in}} + n_{\text{out}}))$.

Z-score. The number of standard deviations a value is from the mean: $z = (x - \bar{x})/s$. Used for outlier detection with a threshold of $|z| > 3$.

Further Reading

On unsupervised learning

- Hastie, T., Tibshirani, R., and Friedman, J. *The Elements of Statistical Learning*. Springer, 2009. Chapters 13 (prototypes and nearest-neighbours), 14 (unsupervised learning), and 8 (model inference and averaging) are directly relevant. Free online at `web.stanford.edu/~hastie/ElemStatLearn/`.

- Bishop, C. *Pattern Recognition and Machine Learning*. Springer, 2006. Chapter 9 (Mixture Models and EM) is the standard reference for the EM algorithm. Chapter 12 (Continuous Latent Variables) covers PCA and factor analysis.

On clustering

- Ester, M., et al. "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise." *KDD*, 1996. The original DBSCAN paper.
- McInnes, L., Healy, J., and Astels, S. "hdbscan: Hierarchical density based clustering." *JOSS*, 2017. The HDBSCAN reference.
- Arthur, D., and Vassilvitskii, S. "k-means++: The Advantages of Careful Seeding." *SODA*, 2007. The K-means++ initialisation paper with its $O(\log K)$ competitive guarantee.

On dimensionality reduction

- Jolliffe, I. *Principal Component Analysis*. Springer, 2002. The definitive PCA reference.
- van der Maaten, L., and Hinton, G. "Visualizing Data using t-SNE." *JMLR*, 2008. The original t-SNE paper.
- McInnes, L., Healy, J., and Melville, J. "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction." *arXiv:1802.03426*, 2018.

On anomaly detection

- Liu, F., Ting, K., and Zhou, Z.-H. "Isolation Forest." *ICDM*, 2008. The original Isolation Forest paper.
- Breunig, M., et al. "LOF: Identifying Density-Based Local Outliers." *SIGMOD*, 2000. The original LOF paper.

On topic modelling and NLP

- Blei, D., Ng, A., and Jordan, M. "Latent Dirichlet Allocation." *JMLR*, 2003. The original LDA paper.
- Grootendorst, M. "BERTopic: Neural topic modeling with a class-based TF-IDF procedure." *arXiv:2203.05794*, 2022.
- Robertson, S., and Zaragoza, H. "The Probabilistic Relevance Framework: BM25 and Beyond." *Foundations and Trends in Information Retrieval*, 2009.

On recommender systems

- Koren, Y., Bell, R., and Volinsky, C. "Matrix Factorization Techniques for Recommender Systems." *Computer*, 2009. The Netflix Prize paper — the definitive introduction to collaborative filtering with matrix factorisation.
- Hu, Y., Koren, Y., and Volinsky, C. "Collaborative Filtering for Implicit Feedback Datasets." *ICDM*, 2008.

On neural networks and deep learning foundations

- Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. MIT Press, 2016. Chapters 6 (Deep Feedforward Networks), 7 (Regularization), and 8 (Optimization) are the standard reference for the material in Lesson 4.8. Free online at `deeplearningbook.org`.

- He, K., et al. “Delving Deep into Rectifiers.” *ICCV*, 2015. The Kaiming initialisation paper.
- Kingma, D., and Ba, J. “Adam: A Method for Stochastic Optimization.” *ICLR*, 2015.
- Ioffe, S., and Szegedy, C. “Batch Normalization: Accelerating Deep Network Training.” *ICML*, 2015.

On Singapore-specific data

- `data.gov.sg` — Singapore government open data portal. HDB resale transactions, retail statistics, and economic indicators.
- Singapore Department of Statistics. Monthly and quarterly reports on retail sales, consumer prices, and economic activity.